

**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

SKRIPSI



IDA BAGUS GEDE BASUDEWA WEDA

NIM.2208561077

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA
JIMBARAN**

2026

**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

SKRIPSI



**IDA BAGUS GEDE BASUDEWA WEDA
NIM.2208561077**

**PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS UDAYANA
JIMBARAN
2026**

SURAT PERNYATAAN KEASLIAN KARYA ILMIAH

Yang bertanda tangan dibawah ini menyatakan bahwa naskah skripsi dngan judul:
**IMPLEMENTASI VARIATIONAL AUTOENCODER DAN
REGULARIZED SINGULAR VALUE DECOMPOSITION PADA SISTEM
REKOMENDASI FILM**

Nama : Ida Bagus Gede Basudewa Weda
NIM : 2208561077
Program Studi : Informatika
E – mail : basudewaweda@gmail.com
Nomor telp/HP : 08179328323
Alamat : Jln Sekar Tunjung XVIIIIB No. 3, Kesiman Kertalangu,
Denpasar, Bali

Belum pernah dipublikasikan dalam dokumen skripsi, jurnal maupun internasional atau dalam prosiding manapun, dan tidak sedang atau akan diajukan untuk publikasi di jurnal atau prosiding manapun. Apabila di kemudian hari terbukti terdapat pelanggaran kaidah-kaidah akademik pada karya ilmiah saya, maka saya bersedia menanggung sanksi-sanksi yang dijatuhkan karena kesalahan tersebut, sebagaimana diatur oleh Peraturan Menteri Pendidikan Nasional Nomor Tahun 17 Tahun 2010 tentang Pencegahan dan Penanggulangan Plagiat di Perguruan Tinggi.

Demikian Surat Pernyataan ini saya buat dengan sesungguhnya untuk dapat digunakan bilaman diperlukan.

Jimbaran, 19 April 2025

Yang membuat pernyataan,

Ida Bagus Gede Basudewa Weda
NIM.2208561077

LEMBAR PENGESAHAN TUGAS AKHIR

Judul : Implementasi *Variational Autoencoder* dan *Regularized Singular Value Decomposition* Pada Sistem Rekomendasi Film

Nama : Ida Bagus Gede Basudewa Weda

NIM : 2208561077

Tanggal Seminar : 7 Mei 2025

Disetujui oleh:

Pembimbing I

Penguji I

Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom.,
M.Kom.
NIP. 197803212005011001

Cokorda Rai Adi Pramatha, ST, MM.PhD
NIP. 197806212006041002

Penguji II

Gst. Ayu Vida Mastrika Giri, S.Kom.,
M.Cs.
NIP. 199006062022032009

Mengetahui,
Program Studi Informatika
FMIPA UNUD
KPS

Dr. Ir. I Ketut Gede Suhartana, S.Kom., M.Kom., IPM., ASEAN.Eng
NIP.197201102008121001

Judul : Implementasi *Variational Autoencoder* dan *Regularized Singular Value Decomposition* Pada Sistem Rekomendasi Film
Nama : Ida Bagus Gede Basudewa Weda
Pembimbing : Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom, M.Kom

ABSTRAK

Keamanan data digital menjadi hal yang sangat penting untuk melindungi informasi sensitif, seperti koperasi simpan pinjam sekolah yang menyimpan data anggota dan transaksi keuangan. Ancaman siber, seperti peretasan, pencurian identitas, dan penyalahgunaan data, dapat berdampak buruk pada operasional dan reputasi koperasi. Algoritma kriptografi digunakan untuk melindungi data tersebut, kriptografi terdiri dari algoritma simetris dan asimetris. Algoritma simetris dan asimetris masing-masing memiliki kelebihan dan kekurangannya. Vigenère cipher, sebagai algoritma simetris, menawarkan kecepatan enkripsi, namun memiliki kelemahan pada keamanan kunci, sedangkan RSA sebagai algoritma asimetris lebih aman, namun dengan proses yang lebih lambat. Untuk mengatasi kelemahan kedua algoritma tersebut, kriptosistem *hybrid* yang menggabungkan kedua algoritma dapat digunakan untuk memberikan lapisan keamanan tambahan. Penelitian ini bertujuan untuk mengevaluasi dan membandingkan hasil enkripsi menggunakan algoritma Vigenere Cipher saja dengan gabungan Vigenère cipher dan RSA serta menguji ketahanan sistem terhadap serangan *brute force*. Hasil penelitian ini diharapkan dapat menunjukkan bahwa kombinasi kedua algoritma ini memberikan perlindungan yang lebih kuat untuk menjaga kerahasiaan informasi yang sensitif.

Kata Kunci: Keamanan Data, Kriptografi, Vigenère Cipher, RSA, Kriptosistem Hybrid, Brute Force.

Judul : *Implementation Of Variational Autoencoder and Regularized Singular Value Decomposition In Film Recommendation System*
Nama : Ida Bagus Gede Basudewa Weda
Pembimbing : Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom, M.Kom

ABSTRAC

Digital data security is very important to protect sensitive information, such as school savings and loan cooperatives that store member data and financial transactions. Cyber threats, such as hacking, identity theft, and data misuse, can have a negative impact on the cooperative's operations and reputation. Cryptographic algorithms are used to protect this data, cryptography consists of symmetric and asymmetric algorithms. Symmetric and asymmetric algorithms each have their advantages and disadvantages. Vigenère cipher, as a symmetric algorithm, offers encryption speed, but has weaknesses in key security, while RSA as an asymmetric algorithm is more secure, but with a slower process. To overcome the weaknesses of both algorithms, a hybrid cryptosystem that combines both algorithms can be used to provide an additional layer of security. This study aims to evaluate and compare the encryption results using the Vigenère Cipher algorithm alone with a combination of Vigenère cipher and RSA and to test the system's resistance to brute force attacks. The results of this study are expected to show that the combination of these two algorithms provides stronger protection to maintain the confidentiality of sensitive information.

Keywords: Data Security, Cryptography, Vigenère Cipher, RSA, Hybrid Cryptosystem, Brute Force.

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa atas berkat dan anugrah-Nya, sehingga saya dapat menyelesaikan penelitian yang berjudul “Analisis Kombinasi *Vigenere Cipher* dengan *Algoritma Rsa* dalam Pengamanan Data Tekstual” dengan tepat waktu. Penelitian ini disusun dalam rangka kegiatan pelaksanaan Tugas Akhir di Program Studi Informatika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Udayana dengan harapan dapat menjadi sumber pedoman dan arahan dalam melaksanakan penelitian di atas.

Penelitian ini telah saya susun dengan semaksimal mungkin dan saya menyadari bahwa penyusunan Tugas Akhir ini tidak akan selesai tanpa adanya bantuan dari berbagai belah pihak. Untuk itu, saya ingin mengucapkan terima kasih kepada :

1. Bapak Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom., M.Kom sebagai Pembimbing I yang telah memberikan arahan dan masukan selama penyusunan tugas akhir ini.
2. Bapak/Ibu dosen yang berada di lingkungan Prodi Informatika Fakultas Matematika Ilmu Pengetahuan dan Alam Universitas Udayana yang telah membimbing dan memberikan masukan maupun saran yang berguna untuk penyempurnaan tugas akhir ini.
3. Orang tua saya yang selalu memberikan dukungan penuh serta doa dalam penyelesaian tugas akhir ini
4. Teman – teman di Program Studi Informatika yang telah memberikan dukungan moral dalam penyelesaian tugas akhir ini.

Terlepas dari itu semua, penelitian ini masih jauh dari kata sempurna. Untuk itu kritik dan saran saya harapkan dari semua pihak demi makin sempurnanya penyusunan penelitian ini.

Jimbaran, 5 Mei 2025

Penulis

Luh Arimas Pertiwi

DAFTAR ISI

HALAMAN JUDUL.....	i
SURAT PERNYATAAN KEASLIAN KARYA ILMIAH	ii
LEMBAR PENGESAHAN TUGAS AKHIR.....	iii
ABSTRAK	iv
<i>ABSTRAC</i>	v
KATA PENGANTAR.....	vi
DAFTAR ISI	vii
DAFTAR TABEL.....	x
DAFTAR GAMBAR	xi
DAFTAR LAMPIRAN	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Batasan Masalah	4
1.4 Tujuan Penelitian	5
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan	5
BAB II TINJAUAN PUSTAKA	5
2.1 Tinjauan Teori	5
2.1.1 Kriptografi	Error! Bookmark not defined.
2.1.2 Vigenere Cipher	Error! Bookmark not defined.
2.1.3 Kriptografi Algoritma RSA	Error! Bookmark not defined.
2.1.4 <i>Black Box Testing</i>	Error! Bookmark not defined.
2.2.5. <i>Brute Force</i>	Error! Bookmark not defined.
2.2 Tinjauan Empiris.....	Error! Bookmark not defined.
2.2.1 Implementasi Kriptografi Algoritma Rsa Untuk Peningkatan Keamanan Database <i>E-Commerce</i> (2019)	Error! Bookmark not defined.
2.2.2 Pengamanan File Dengan Algoritma Kriptografi Rivest Shamir Adleman (Rsa) Studi Kasus Smp Muhammadiyah 4 (2021)	Error! Bookmark not defined.

2.2.3 Kombinasi Algoritma Caesar Cipher Dan Algoritma Rsa Untuk Pengamanan File Dokumen Dan Pesan Teks (2018)	Error! Bookmark not defined.
2.2.4 Pengamanan File Teks Menggunakan Algoritma Rsa – Luc Dan Algoritma Zig-Zag Dalam <i>Hybrid Crypto</i> Sistem (2022)	Error! Bookmark not defined.
2.2.5 Penerapan Kriptografi Vigenere Cipher Pada Keamanan Data Pesan Teks Berbasis Website (2022)	Error! Bookmark not defined.
2.2.6. Implementasi Kriptografi Vigenere Cipher Dengan Php (2020).	Error! Bookmark not defined.
2.2.7. Implementasi Kombinasi Vigenere Cipher dengan RSA dalam Kriptosistem Hibrida untuk Keamanan teks (2020)	Error! Bookmark not defined.
BAB III ANALISIS DAN PERANCANGAN SISTEM	18
3.1 Metode Pengembangan sistem	Error! Bookmark not defined.
3.1.1 Data dan Metode Pengumpulan Data	Error! Bookmark not defined.
3.1.2 Desain Sistem	Error! Bookmark not defined.
3.1.2.1 Gambaran Umum Sistem	Error! Bookmark not defined.
3.1.2.2 Pengembangan Algoritma Vigenere Cipher dengan Algoritma RSA	Error! Bookmark not defined.
3.1.2.3 Perancangan Antarmuka	Error! Bookmark not defined.
3.1.3 Implementasi Sistem	Error! Bookmark not defined.
3.1.4 Desain Evaluai Sistem/Metode	Error! Bookmark not defined.
3.1.4.1 Pengujian hasil enkripsi dan dekripsi	Error! Bookmark not defined.
3.1.4.2 Pengujian <i>Blackbox</i>	Error! Bookmark not defined.
3.1.4.3 Pengujian <i>Brute Force</i>	Error! Bookmark not defined.
BAB IV HASIL DAN PEMBAHASAN	29
4.1 Proses Pengumpulan Data	Error! Bookmark not defined.
4.2 Implementasi Sistem	Error! Bookmark not defined.
4.2.1 Hasil Desain Sistem	Error! Bookmark not defined.

4.2.2 Implementasi Kombinasi Algoritma Vigenere Cipher dengan Algoritma RSA	Error! Bookmark not defined.
4.3 Evaluasi Sistem.....	Error! Bookmark not defined.
4.3.1 Hasil Enkripsi dan dekripsi.....	Error! Bookmark not defined.
4.3.2 Pengujian Blackbox.....	Error! Bookmark not defined.
4.3.3 Pengujian <i>Brute Force</i>	Error! Bookmark not defined.
BAB V KESIMPULAN DAN SARAN.....	Error! Bookmark not defined.
5.1 Kesimpulan	Error! Bookmark not defined.
5.2 Saran	Error! Bookmark not defined.
DAFTAR PUSTAKA	Error! Bookmark not defined.
LAMPIRAN.....	Error! Bookmark not defined.

DAFTAR TABEL

Tabel 4. 1. Hasil Enkripsi dan Dekripsi File Docx	Error! Bookmark not defined.
Tabel 4. 2. Hasil Enkripsi dan Dekripsi File txt	Error! Bookmark not defined.
Tabel 4. 3. Pengujian hasil Enkripsi dan Dekripsi Kunci dibangkitkan Sistem	Error! Bookmark not defined.
Tabel 4. 4. Pengujian Black Box Proses Enkripsi	Error! Bookmark not defined.
Tabel 4. 5. Pengujian Black Box Proses Dekripsi	Error! Bookmark not defined.
Tabel 4. 6. Pengujian Brute Force pada Algoritma Vigenere Cipher	 Error! Bookmark not defined.
Tabel 4. 7. Pengujian Brute Force Kunci RSA	Error! Bookmark not defined.
Tabel 4. 8. Pengujian Brute Force Kunci Vigenere dengan RSA 8 Bit	 Error! Bookmark not defined.
Tabel 4. 9. Pengujian Brute Force Kunci Vigenere dengan RSA 16 Bit	 Error! Bookmark not defined.
Tabel 4. 10. Pengujian Brute Force Kunci Vigenere dengan RSA 32 Bit	 Error! Bookmark not defined.
Tabel 4. 11. Pengujian Brute Force Kunci Vigenere dengan RSA 64 Bit	 Error! Bookmark not defined.
Tabel 4. 12. Pengujian Brute Force Kunci Vigenere dengan RSA 64 bit yang Diolah Sistem	Error! Bookmark not defined.

DAFTAR GAMBAR

Gambar 2. 1. Proses Sederhana Kriptografi.....	Error! Bookmark not defined.
Gambar 2. 2. Skema kriptografi simetris	Error! Bookmark not defined.
Gambar 2. 3. Skema kriptografi Asimetris.....	Error! Bookmark not defined.
Gambar 2. 4. Tabel Pemetaan Vigenere Cipher	Error! Bookmark not defined.
Gambar 2. 5. Urutan Konversi Huruf Alfabet.....	Error! Bookmark not defined.
Gambar 3. 1. Metode Penelitian.....	18
Gambar 3. 2. Gambaran Umum Sistem	Error! Bookmark not defined.
Gambar 3. 3. Alur Umum Enkripsi dan Dekripsi...	Error! Bookmark not defined.
Gambar 3. 4. Alur Enkripsi	Error! Bookmark not defined.
Gambar 3. 5. Alur Dekripsi.....	Error! Bookmark not defined.
Gambar 3. 6. Tampilan Beranda (Home)	Error! Bookmark not defined.
Gambar 3. 7. Menu Enkripsi.....	Error! Bookmark not defined.
Gambar 3. 8. Menu Dekripsi.....	Error! Bookmark not defined.
Gambar 4. 1. Tampilan Beranda (Home).....	Error! Bookmark not defined.
Gambar 4. 2. Menu Enkripsi.....	Error! Bookmark not defined.
Gambar 4. 3. Error kunci RSA.....	Error! Bookmark not defined.
Gambar 4. 4. Menu Dekripsi.....	Error! Bookmark not defined.
Gambar 4. 5 Penggalan Kode Inputan Proses Enkripsi	Error! Bookmark not defined.
Gambar 4. 6. Penggalan Kode Pembentukan Kunci Vigenere Cipher.....	Error! Bookmark not defined.
Gambar 4. 7 Penggalan Kode Pemangginlan Fungsi RSA Key Generation..	Error! Bookmark not defined.
Gambar 4. 8 Penggalan Kode Pembentukan Kunci RSA	Error! Bookmark not defined.
Gambar 4. 9 Penggalan Kode Enkripsi Vigenere Cipher	Error! Bookmark not defined.
Gambar 4. 10. Penggalan Kode Enkripsi RSA	Error! Bookmark not defined.

Gambar 4. 11 Penggalan Kode Proses Enkripsi Kombinasi Vigenere Cipher dengan RSA.....**Error! Bookmark not defined.**

Gambar 4. 12 Penggalan Kode Inputan Proses Dekripsi**Error! Bookmark not defined.**

Gambar 4. 13 Penggalan Kode Proses Dekripsi Vigenere Cipher **Error! Bookmark not defined.**

Gambar 4. 14 Penggalan Kode Proses Dekripsi Algoritma RSA**Error! Bookmark not defined.**

Gambar 4. 15 Penggalan Kode Proses Dekripsi Kombinasi Vigenere dengan Algoritma RSA.....**Error! Bookmark not defined.**

Gambar 4. 16. Estimasi Waktu Brute Force Algoritma Vigenere Cipher **Error! Bookmark not defined.**

Gambar 4. 17. Estimasi Waktu Brute Force Kunci RSA**Error! Bookmark not defined.**

Gambar 4. 18. Estimasi Waktu Brute Force Kunci Vigenere dengan RSA 8 Bit**Error! Bookmark not defined.**

Gambar 4. 19. Estimasi Waktu Brute Force Kunci Vigenere dengan RSA 16 Bit**Error! Bookmark not defined.**

Gambar 4. 20. Estimasi Waktu Brute Force Kunci Vigenere dengan RSA 32 Bit**Error! Bookmark not defined.**

Gambar 4. 21 Estimasi Waktu Brute Force Kunci Vigenere dengan RSA 64 Bit**Error! Bookmark not defined.**

DAFTAR LAMPIRAN

- Lampiran 1. File hasil enkripsi
- Lampiran 2. File hasil dekripsi
- Lampiran 3. Data asli (plainteks) file docx
- Lampiran 4. Hasil enkripsi algoritma vigenere cipher file docx
- Lampiran 5. Hasil enkripsi algoritma vigenere cipher dengan algoritma RSA file docx
- Lampiran 6. Hasil dekripsi file docx
- Lampiran 7. Data asli (plainteks) file txt.
- Lampiran 8. Hasil enkripsi algoritma vigenere cipher file txt.
- Lampiran 9. Hasil enkripsi algoritma vigenere cipher dengan algoritma RSA file txt
- Lampiran 10. Hasil dekripsi file txt
- Lampiran 11. Data asli (plainteks) file.xlsx
- Lampiran 12. Hasil enkripsi algoritma vigenere cipher file.xlsx
- Lampiran 13. Hasil enkripsi algoritma vigenere cipher dengan algoritma RSA file.xlsx
- Lampiran 14. Hasil dekripsi file txt
- Lampiran 15. Source Code sistem

BAB I

PENDAHULUAN

1.1 Latar Belakang

Industri perfilman mengalami peningkatan yang signifikan pada jumlah film yang diproduksi setelah terjadinya penurunan produksi yang disebabkan oleh terjadinya pandemi COVID-19 pada tahun 2020. Pada tahun 2021 produksi film meningkat sebesar 40% dan pada tahun 2022 meningkat sebesar 15%. Keuntungan yang diraup dari Global box office juga meningkat pada tahun 2022 meningkat sebesar 21,8% dan pada tahun 2023 meningkat sebesar 29,4% (Hancock et al., 2024). Peningkatan kuantitas film yang diproduksi dan keuntungan yang didapatkan akan memperbanyak film yang tersedia. Banyaknya film yang tersedia akan memberikan pilihan yang banyak kepada penonton, tetapi juga memberikan tantangan yaitu menemukan film yang sesuai dengan preferensi mereka. Sistem rekomendasi dapat digunakan untuk mengatasi tantangan ini.

Sistem rekomendasi merupakan sistem yang berfungsi untuk memperkirakan dan memberikan informasi yang sekiranya menarik untuk seorang pengguna. Sistem rekomendasi menggunakan berbagai metode untuk memberikan saran produk berdasarkan preferensi dan kebutuhan pengguna seperti content-based filtering, collaborative filtering, dan metode hybrid. Metode content-based filtering bekerja dengan menganalisis atribut atau fitur dari item-item yang disukai oleh pengguna dan kemudian merekomendasikan item lain yang memiliki kemiripan atribut (Ayu et al., 2021). Metode collaborative filtering dapat dibagi menjadi dua kategori yaitu memory-based dan model-based. Kategori memory-based menggunakan tingkat similaritas untuk memberikan prediksi dengan mencari asosiasi antara pengguna dengan item (Rajput et al., 2024). Kategori memory-based dapat dikategorikan lagi menjadi item-based dan user-based. Item-based menganalisis bagaimana pengguna menilai suatu item dan mencari item dengan penilai yang mirip. User-based mencari pengguna yang mirip berdasarkan penilaian yang diberikan dan merekomendasikan item yang pengguna mirip sudah sukai ke pengguna target. Model-based menggunakan matriks pengguna-item

sebagai data untuk membangun dan melatih sebuah model yang dapat memprediksi penilaian pengguna terhadap suatu item.

Salah satu tantangan utama dalam sistem rekomendasi adalah data sparsity. Data sparsity merujuk pada situasi di mana interaksi antara pengguna dan item sangat sedikit atau jarang. Dalam sistem rekomendasi berbasis matriks, ini berarti matriks pengguna-item memiliki banyak nilai kosong atau tidak diketahui. Hal ini menyulitkan model untuk menemukan pola yang cukup kuat untuk memberikan rekomendasi yang akurat. Misalnya, dalam layanan streaming dengan ribuan film, kebanyakan pengguna mungkin hanya menonton atau memberikan rating pada sebagian kecil dari film yang tersedia, menyebabkan matriks menjadi sangat jarang terisi.

Penelitian terkait sistem rekomendasi telah banyak dilakukan untuk mengatasi masalah data sparsity dan meningkatkan akurasi prediksi. Rajput et al. (2024) menggunakan penggabungan standard autoencoder dan faktorisasi matriks dengan singular value decomposition (SVD) untuk mengatasi masalah sparsity dalam sistem rekomendasi multi-kriteria, dengan hasil MAE sebesar 0.3350 dan RMSE sebesar 0.4897. Wibisono et al. (2021) mengembangkan sistem rekomendasi menggunakan item-based collaborative filtering dengan SVD yang terbukti mampu mereduksi dimensi matriks rating dan mengurangi noise pada data sparse, dengan hasil MAE sebesar 1.2752 dan RMSE sebesar 1.4882. Selain itu, Mu'afa dan Baizal (2022) menunjukkan bahwa penggabungan SVD dengan metode lain mampu meningkatkan akurasi prediksi rating dengan RMSE sebesar 0.835 dibandingkan 0.896 tanpa SVD. Penelitian-penelitian ini secara konsisten menunjukkan potensi penggabungan faktorisasi matriks berbasis SVD dengan metode lain untuk meningkatkan performa sistem rekomendasi. Namun, seluruh penelitian tersebut menggunakan standard autoencoder, bukan Variational Autoencoder (VAE), dan SVD tanpa regularisasi sehingga masih terdapat gap penelitian yang belum dieksplorasi.

Berdasarkan gap tersebut, penelitian ini mengajukan pendekatan yang menggabungkan Variational Autoencoder (VAE) dan Regularized Singular Value Decomposition (RSVD). VAE dipilih karena memiliki karakteristik yang sesuai

untuk sistem rekomendasi, sifat Bayesian-nya memungkinkan VAE menangani data sparse dan kompleks seperti data interaksi pengguna-item secara lebih tepat karena ketidakpastian diperhitungkan dalam proses inferensi, dan VAE mempelajari representasi laten sebagai distribusi probabilistik sehingga menghasilkan latent space yang lebih terstruktur dibandingkan standard autoencoder (Liang et al., 2024; Bobadilla et al., 2023). Namun, pelatihan VAE pada data collaborative filtering berisiko mengalami posterior collapse, yaitu kondisi di mana model mengabaikan struktur ruang laten dan hanya mengandalkan decoder sehingga representasi laten menjadi tidak informatif (Wang et al., 2025). Untuk mengatasi hal ini, penelitian ini menerapkan KL annealing yaitu sebuah teknik yang menaikkan bobot β pada KL divergence secara bertahap selama pelatihan agar model terlebih dahulu mempelajari rekonstruksi sebelum tekanan regularisasi diberikan sepenuhnya (Liang et al., 2024). Di sisi lain, SVD standar pada sistem rekomendasi sering kali menemui kendala ketika dihadapkan pada data yang sangat sparse. SVD standar cenderung mengalami overfitting karena model berusaha menghafal data latih sehingga menghasilkan generalisasi yang buruk pada data yang belum pernah dilihat sebelumnya. Penerapan regularisasi pada RSVD mengatasi kelemahan ini dengan memberikan penalti pada parameter latent factor pengguna dan item, yang secara signifikan mengurangi risiko overfitting dan membuat model lebih stabil serta akurat dalam memprediksi rating pada matriks yang jarang terisi (Darma Putra & Santiyasa, 2025). Kombinasi spesifik VAE dan RSVD ini belum pernah dieksplorasi dalam penelitian sebelumnya, sehingga penelitian ini berkontribusi dalam mengisi gap tersebut.

Pendekatan hybrid yang menggabungkan dua model dengan karakteristik berbeda terbukti mampu mengatasi keterbatasan masing-masing model secara individual dalam sistem rekomendasi, karena setiap model dapat saling melengkapi kelemahan yang lain (Sami et al., 2024). Dalam penelitian ini, VAE dan RSVD memiliki karakteristik yang saling komplementer, VAE unggul dalam menangkap pola preferensi pengguna yang kompleks dan non-linear melalui representasi probabilistik di ruang laten (Liang et al., 2024; Bobadilla et al., 2023), namun prediksinya dapat kurang stabil pada data yang sangat sparse karena bergantung

pada sampling dari distribusi laten. Sebaliknya, RSVD menghasilkan prediksi yang lebih stabil melalui dekomposisi bias dan faktor laten yang terregularisasi (Darma Putra & Santiyasa, 2025), namun terbatas dalam menangkap hubungan non-linear antar pengguna dan item. Dengan menggabungkan prediksi keduanya melalui weighted average ensemble penelitian ini bertujuan memanfaatkan kekuatan representasi probabilistik VAE sekaligus kestabilan prediksi RSVD, sehingga diharapkan menghasilkan sistem rekomendasi yang lebih akurat dibandingkan masing-masing model secara mandiri.

Berdasarkan hal tersebut, penelitian ini bertujuan untuk mengembangkan dan mengevaluasi sistem rekomendasi film yang menggabungkan VAE dan RSVD menggunakan dataset MovieLens 100k, serta mengukur performanya menggunakan metrik *mean squared error* (MSE), *root mean squared error* (RMSE), dan *mean absolute error* (MAE).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijabarkan, rumusan masalah pada penelitian ini adalah sebagai berikut:

- a. Bagaimana performa sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition* dalam menghasilkan rekomendasi?
- b. Bagaimana pengembangan model sistem rekomendasi *variational autoencoder* dan *regularized singular value decomposition*?
- c. Bagaimana model yang optimal untuk sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition*?
- d. Bagaimana kinerja implementasi sistem rekomendasi yang menggunakan *variational autoencoder* dan *regularized singular value decomposition*?

1.3 Batasan Masalah

Terdapat beberapa batasan permasalahan dalam penelitian ini:

- a. Data didapatkan secara sekunder dari situs web grouplens.org yang berjudul “movielens 100k”.
- b. Pengguna tidak dapat menonton film secara langsung pada sistem.

- c. Sistem hanya dapat memberikan rekomendasi item yang terdapat pada data.

1.4 Tujuan Penelitian

Berdasarkan latar belakang dan rumusan masalah yang telah dijabarkan, tujuan dari penelitian ini adalah sebagai berikut:

- d. Mengetahui performa sistem rekomendasi yang menggunakan *variational autoencoder* dan *regularized singular value decomposition* dalam menghasilkan rekomendasi.
- e. Mengetahui bagaimana pengembangan model sistem rekomendasi *variational autoencoder* dan *regularized singular value decomposition*
- f. Mengetahui model yang optimal untuk sistem rekomendasi dengan *variational autoencoder* dan *regularized singular value decomposition*.
- g. Mengetahui kinerja implementasi sistem rekomendasi yang menggunakan *variational autoencoder* dan *regularized singular value decomposition*.

1.5 Manfaat Penelitian

Manfaat dari hasil penelitian ini adalah:

- a. Bagi penonton, dapat dengan mudah mendapatkan rekomendasi yang sesuai dengan preferensi mereka.
- b. Bagi pembuat film, Film yang dibuat dapat direkomendasikan kepada pengguna sesuai dengan preferensi mereka.

1.6 Sistematika Penulisan

Untuk memberikan gambaran yang jelas dan menyeluruh mengenai penyusunan Tugas Akhir ini, maka penulisan dibagi menjadi lima bab dengan sistematika sebagai berikut:

BAB I PENDAHULUAN

Bab ini menguraikan kerangka dasar penelitian yang meliputi latar belakang masalah mengenai data sparsity pada sistem rekomendasi film, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan yang digunakan dalam penyusunan tugas akhir.

BAB II TINJAUAN PUSTAKA

Bab ini membahas landasan teori dan tinjauan empiris dari penelitian-penelitian terdahulu yang relevan. Landasan teori yang diuraikan mencakup konsep dasar sistem rekomendasi, *Collaborative Filtering*, *Variational Autoencoder* (VAE), *Regularized Singular Value Decomposition* (RSVD), serta metrik evaluasi pengujian (seperti MSE, RMSE, dan MAE). Bab ini menjadi dasar pijakan untuk perancangan model yang diusulkan.

BAB III METODOLOGI PENELITIAN

Bab ini menjelaskan tahapan, analisis, dan perancangan yang dilakukan dalam penelitian. Pembahasan di dalamnya meliputi desain sistem secara umum, teknik pengumpulan data sekunder (dataset MovieLens 100k), perancangan model (preprocessing data, pengembangan model VAE, dan dekomposisi dengan RSVD), perancangan antarmuka sistem rekomendasi berbasis web, serta rancangan skenario evaluasi kinerja sistem.

BAB IV HASIL DAN PEMBAHASAN

Bab ini memaparkan secara detail hasil dari implementasi sistem dan pengujian model yang telah dirancang pada bab sebelumnya. Pembahasan difokuskan pada analisis performa sistem rekomendasi hasil penggabungan arsitektur VAE dan RSVD, perbandingan metrik evaluasi untuk mengukur akurasi prediksi, serta hasil implementasi antarmuka (frontend dan backend) sistem rekomendasi film.

BAB V SIMPULAN DAN SARAN

Bab ini berisi kesimpulan akhir yang ditarik dari hasil analisis dan pembahasan untuk menjawab rumusan masalah penelitian. Selain itu, pada bab ini juga dikemukakan saran-saran yang konstruktif untuk mengatasi keterbatasan yang ada serta sebagai bahan pertimbangan bagi pengembangan sistem atau penelitian sejenis di masa yang akan datang.

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Teori

2.1.1 Sistem Rekomendasi

Sistem rekomendasi merupakan sistem yang berfungsi untuk memperkirakan dan memberikan informasi yang sekiranya menarik untuk seorang pengguna (Nugroho & Rahayu, 2020). Sistem rekomendasi bekerja dengan menganalisis data pengguna, seperti riwayat pembelian, tontonan, dan interaksi mereka dengan platform digital. Berdasarkan data tersebut, sistem rekomendasi dapat memprediksi apa yang disukai dan dibutuhkan pengguna, dan kemudian memberikan rekomendasi yang dipersonalisasi. Rekomendasi yang diberikan oleh sistem dapat digunakan oleh pengguna untuk mempertimbangkan apa yang harus mereka lakukan, seperti membeli barang atau menonton pertunjukan. Sistem ini banyak digunakan dalam berbagai industri, seperti *e-commerce*, *media streaming*, media sosial, dan lainnya.

2.1.2 Collaborative Filtering

Collaborative filtering adalah teknik yang digunakan dalam sistem rekomendasi untuk memberikan saran item kepada pengguna berdasarkan preferensi pengguna lain yang serupa (Sutjiningtyas et al., 2022). Sederhananya, sistem ini mencari pengguna yang memiliki selera yang sama dengan pengguna yang ingin mendapatkan rekomendasi, dan kemudian merekomendasikan item yang disukai oleh pengguna-pengguna tersebut. Contohnya, di situs web *e-commerce*, *collaborative filtering* dapat digunakan untuk merekomendasikan produk kepada pengguna berdasarkan produk yang telah mereka beli atau lihat sebelumnya, serta produk yang dibeli oleh pengguna lain yang memiliki profil serupa.

Collaborative filtering dapat dibagi menjadi dua kategori yaitu *memory-based* dan *model-based*. Kategori *memory-based* menggunakan tingkat similaritas untuk memberikan prediksi dengan mencari asosiasi antara pengguna dengan *item* (Rajput et al., 2024). Kategori *memory-based* dapat dikategorikan lagi menjadi *item-based* dan *user-based*. *Item-based* menganalisis bagaimana pengguna menilai

suatu *item* dan mencari *item* dengan penilai yang mirip. *User-based* mencari pengguna yang mirip berdasarkan penilaian yang diberikan dan merekomendasikan item yang pengguna mirip sudah sukai ke pengguna target. *Model-based* menggunakan matriks pengguna-item sebagai data untuk membangun dan melatih sebuah model yang dapat memprediksi penilaian pengguna terhadap suatu item.

2.1.3 Variational Autoencoder (VAE)

Variational Autoencoder (VAE) adalah model generatif berbasis jaringan saraf tiruan yang mempelajari representasi laten dari data masukan sebagai distribusi probabilistik, bukan sebagai titik tetap di ruang laten (Liang et al., 2024). Berbeda dengan autoencoder standar yang memetakan setiap input ke satu titik laten, VAE memetakan input ke parameter distribusi Gaussian, yaitu rata-rata μ dan log-varians $\log(\sigma^2)$, sehingga ruang laten menjadi kontinu dan terstruktur. VAE memodelkan data sebagai:

$$p(x, z) = p(x|z)p(z) \quad (1)$$

Keterangan:

- x = data masukan
- z = representasi laten
- $p(x|z)$ = probabilitas x diberikan z
- $p(z)$ = prior dari z , diasumsi sebagai distribusi Gaussian standar $N(0,1)$

Karena posterior sejati $p(z|x)$ bersifat *intractable*, VAE mengaproksimasi posterior tersebut dengan distribusi variasional:

$$q(z|x) = N(z|\mu(x), \sigma(x)) \quad (2)$$

Keterangan:

- $\mu(x)$ = rata-rata yang dihasilkan *encoder*
- $\sigma(x)$ = varians yang dihasilkan *encoder*

Fungsi objektif VAE adalah *Evidence Lower Bound* (ELBO):

$$\mathcal{L} = E_{q(z|x)}[\log p(x|z)] - \beta \cdot KL(q(z|x)||p(z)) \quad (3)$$

Keterangan:

- $E_{q(z|x)}[\log p(x|z)]$ = *reconstruction loss*
- $\beta \cdot KL(q(z|x)||p(z))$ = KL divergence

Bagian pertama adalah *reconstruction loss* yang mengukur seberapa baik *decoder* merekonstruksi data dari z , dan bagian kedua adalah *KL divergence* yang meregularisasi distribusi laten agar mendekati prior Gaussian. *Hyperparameter* β mengontrol bobot antara dua komponen ini (Wang et al., 2025).

Untuk kasus *prior* Gaussian standar $\mathcal{N}(0, I)$, *KL divergence* antara distribusi posterior $q(z | x)$ dan prior $p(z)$ dapat dihitung sebagai:

$$L_{KL} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (4)$$

Keterangan:

- d = dimensi ruang laten
- μ_j = rata-rata dimensi ke- j dari distribusi *posterior*
- σ_j^2 = varians dimensi ke- j dari distribusi *posterior*

Pada penelitian ini, *reconstruction loss* dihitung hanya pada *rating* yang tersedia, mengabaikan sel kosong dalam matriks interaksi yang belum diberi *rating*. Pendekatan ini disebut *masked reconstruction loss* dan diformulasikan sebagai:

$$L_{recon} = (\sum_{\{(u,i) \in \Omega\}} (r_{ui} - \hat{r}_{ui})^2) / |\Omega| \quad (5)$$

Keterangan:

- Ω = himpunan pasangan *user item* yang memiliki *rating*
- r_{ui} = *rating* asli
- $\hat{r}_{ui}$ = prediksi rekonstruksi *decoder*

Pendekatan *masked loss* mencegah model belajar dari nilai kosong yang bukan merupakan preferensi nyata pengguna, sehingga representasi laten yang dihasilkan lebih akurat mencerminkan pola interaksi yang sebenarnya. Total loss VAE pada penelitian ini adalah:

$$L_{VAE} = L_{recon} + \beta \cdot KL(q(z|x) || p(z)) \quad (6)$$

Untuk mencegah *posterior collapse* yaitu kondisi di mana model mengabaikan z dan hanya mengandalkan decoder (Wang et al., 2025), nilai β dinaikkan secara bertahap dari 0 menuju nilai target selama pelatihan menggunakan teknik KL annealing (Liang et al., 2024).

Sama seperti *autoencoder* pada umumnya, VAE memiliki tiga komponen utama (Liang et al., 2024):

- Lapisan *Encoder*

Lapisan *Encoder* berfungsi mengubah data masukan menjadi parameter distribusi laten. *Encoder* menerima vektor rating pengguna dan menghasilkan μ dan $\log(\sigma^2)$ dari distribusi Gaussian laten.

- Lapisan *Sampling*

Lapisan *Sampling* memungkinkan *backpropagation* melalui proses sampling menggunakan *reparameterization trick*. Sampel z diambil dari distribusi Gaussian menggunakan rumus:

$$z = \mu + \sigma \cdot \varepsilon \quad (7)$$

Keterangan:

- $\varepsilon \sim N(0, I)$ = sampel acak dari distribusi normal standar
- μ = rata-rata distribusi
- σ = standar deviasi distribusi

- Lapisan *Decoder*

Lapisan Decoder merekonstruksi data dari representasi laten z menggunakan lapisan *dense* dengan fungsi aktivasi *sigmoid* untuk menghasilkan prediksi rating dalam rentang 0 - 1.

2.1.4 Regularized Singular Value Decomposition (RSVD)

Singular Value Decomposition (SVD) merupakan teknik faktorisasi matriks yang memecah sebuah matriks menjadi komponen-komponen yang lebih sederhana. Dalam konteks sistem rekomendasi, SVD digunakan untuk mendekomposisi matriks rating pengguna-item menjadi representasi laten berdimensi rendah yang menangkap pola preferensi tersembunyi (Alshbanat et al., 2025). Matriks A berukuran $m \times n$ yang didekomposisi dengan SVD dinyatakan sebagai:

$$A = U\Sigma V^T \quad (8)$$

Keterangan:

- U = matriks ortogonal berukuran $m \times r$ yang memuat vektor eigen dari $M^T M$
- Σ = matriks diagonal berukuran $r \times r$ berisi nilai-nilai singular non-negatif
- V^T = matriks ortogonal berukuran $r \times n$ yang memuat vektor eigen dari MM^T

Namun, SVD klasik tidak dapat diterapkan langsung pada matriks rating yang sparse karena memerlukan matriks lengkap. Untuk mengatasi hal ini sekaligus mencegah *overfitting*, diterapkan regularisasi L2 sehingga menghasilkan *Regularized Singular Value Decomposition* (RSVD). Selain itu, implementasi RSVD pada sistem rekomendasi diperkaya dengan *bias terms* berupa rata-rata global (μ), bias pengguna (b_u), dan bias item (b_i) untuk menangkap kecenderungan rating yang bersifat sistematis di luar faktor laten (Alshbanat et al., 2025). Prediksi rating dari pengguna u terhadap item i diformulasikan sebagai:

$$\hat{r} = \mu + b_u + b_i + (U\Sigma V^T)_{ui} \quad (9)$$

Keterangan:

- μ = rata-rata *rating* global
- b_u = bias pengguna u
- b_i = bias *item* i
- $(U\Sigma V^T)_{ui}$ = komponen faktor laten

Fungsi *loss* yang dioptimalkan menggunakan regularisasi L2 atau *Ridge Regularization* adalah:

$$Loss = \sum_{i,j} (Z_{ij} - \hat{r}_{ij})^2 + \lambda (\|U\|_F^2 + \|V\|_F^2) \quad (10)$$

Keterangan:

- Z = matriks rating asli
- λ = bobot penalti regularisasi
- $\|X\|_F^2$ = kuadrat norma *Frobenius* dari x

Norma Frobenius dari matriks A berukuran $m \times n$ dihitung sebagai:

$$\|A\|_F = \sqrt{(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2)} \quad (11)$$

Keterangan:

- a_{ij} = elemen baris ke- i kolom ke- j dari matriks A

Untuk mengoptimalkan fungsi *loss*, digunakan metode *Stochastic Gradient Descent* (SGD) yang memperbarui parameter secara iteratif menggunakan informasi gradien dari satu data rating pada setiap langkah pembaruan (Alshbanat et al., 2025). Berbeda dengan *gradient descent* biasa yang menghitung gradien dari seluruh data sekaligus, SGD melakukan pembaruan hanya pada pasangan (u, i) yang memiliki rating aktual ($Z[u,i] > 0$), sehingga sel-sel kosong dalam matriks

sparse tidak mempengaruhi gradien. Aturan pembaruan untuk setiap parameter adalah:

$$U[u, k] \leftarrow U[u, k] + \eta \cdot (e_{ui} \cdot \Sigma[k, k] \cdot V[i, k] - \lambda \cdot U[u, k]) \quad (12)$$

$$V[i, k] \leftarrow V[i, k] + \eta \cdot (e_{ui} \cdot \Sigma[k, k] \cdot U[u, k] - \lambda \cdot V[i, k]) \quad (13)$$

$$\Sigma[k, k] \leftarrow \Sigma[k, k] + \eta \cdot (e_{ui} \cdot U[u, k] \cdot V[i, k]) \quad (14)$$

dimana η adalah *learning rate* dan error rekonstruksi e_{ui} didefinisikan sebagai:

$$e_{ui} = Z[u, i] - \hat{r}_{ui} \quad (15)$$

Terlihat pada rumus (11) dan (12) bahwa pembaruan U dan V mengandung suku penalti regularisasi λ , sedangkan pada rumus (13) pembaruan Σ tidak mengandung suku penalti tersebut. Keputusan mengenai penerapan regularisasi pada Σ serta strategi inisialisasinya dibahas lebih lanjut pada bagian metodologi.

2.1.5 Sistem Rekomendasi Hibrid

Sistem rekomendasi hibrid merupakan pendekatan yang menggabungkan dua atau lebih teknik rekomendasi untuk memanfaatkan kelebihan masing-masing teknik secara bersamaan (Sami et al., 2024). Pendekatan ini dilatarbelakangi oleh kenyataan bahwa setiap teknik rekomendasi tunggal memiliki keterbatasan tersendiri. Collaborative filtering rentan terhadap masalah cold-start dan data sparsity, sedangkan metode berbasis konten bergantung pada ketersediaan atribut item yang lengkap. Dengan menggabungkan beberapa teknik, sistem rekomendasi hibrid dapat saling menutupi kelemahan masing-masing sehingga menghasilkan prediksi yang lebih akurat (Sami et al., 2024).

Salah satu strategi hibridisasi yang umum digunakan adalah weighted hybridization, di mana prediksi akhir diperoleh dari kombinasi linear tertimbang (weighted linear combination) dari prediksi yang dihasilkan oleh masing-masing model penyusun. Pada strategi ini, setiap model diberi bobot yang mencerminkan proporsi kontribusinya terhadap prediksi akhir, dengan total seluruh bobot bernilai satu. Sami et al. (2024) menggunakan strategi ini untuk menggabungkan prediksi dari beberapa model rekomendasi, dengan prediksi akhir dirumuskan sebagai kombinasi linear dari skor prediksi masing-masing model yang dikalikan dengan bobot yang sesuai. Untuk sistem hibrid yang menggabungkan dua model, formula prediksi *weighted hybridization* dapat dituliskan sebagai:

$$\hat{r}_{ui} = \alpha \cdot \hat{r}_{ui}^A + \beta \cdot \hat{r}_{ui}^B \quad (16)$$

dengan *constraint*:

$$\alpha + \beta = 1, \alpha, \beta \in [0,1] \quad (17)$$

Keterangan:

- $\hat{r}_{ui}$ = prediksi *rating* pengguna u terhadap item i
- $\hat{r}_{ui}^A$ = prediksi *rating* model A untuk pengguna u terhadap item i
- $\hat{r}_{ui}^B$ = prediksi *rating* model B untuk pengguna u terhadap item i
- α = bobot model A
- β = bobot model B

Nilai bobot α dan β dapat ditentukan secara empiris melalui pencarian pada rentang 0 – 1 yang meminimalkan metrik kesalahan prediksi pada data validasi. Pendekatan ini memastikan proporsi kontribusi masing-masing model ditentukan berdasarkan performa aktual pada data, sehingga kombinasi yang dihasilkan bersifat adaptif terhadap karakteristik dataset yang digunakan.

2.1.6 Mean Absolute Error, Mean Squared Error, Root Mean Squared Error

Evaluasi performa sistem rekomendasi berbasis prediksi rating dilakukan menggunakan metrik kesalahan yang mengukur selisih antara nilai rating aktual dengan nilai rating yang diprediksi oleh model. Tiga metrik yang digunakan dalam penelitian ini adalah *Mean Absolute Error* (MAE), *Mean Squared Error* (MSE), dan *Root Mean Squared Error* (RMSE) (Bobadilla et al., 2023).

Mean Absolute Error (MAE) mengukur rata-rata nilai absolut dari selisih antara *rating* aktual dan *rating* prediksi. MAE memberikan gambaran langsung seberapa jauh prediksi model menyimpang dari nilai sebenarnya dalam satuan yang sama dengan skala *rating*. MAE diformulasikan sebagai:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (18)$$

Mean Squared Error (MSE) mengukur rata-rata kuadrat selisih antara *rating* aktual dan *rating* prediksi. Pengkuadratan selisih menyebabkan MSE memberikan penalti yang lebih besar terhadap kesalahan prediksi yang jauh dari nilai aktual dibandingkan MAE. MSE diformulasikan sebagai:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (19)$$

Root Mean Squared Error (RMSE) merupakan akar kuadrat dari MSE. RMSE mengembalikan nilai kesalahan ke satuan yang sama dengan skala *rating* asli sehingga lebih mudah diinterpretasikan, sekaligus mempertahankan sifat MSE yang memberikan penalti lebih besar pada kesalahan besar. RMSE diformulasikan sebagai:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (20)$$

Keterangan:

- n = jumlah pasangan rating aktual dan prediksi yang dievaluasi
- y_i = nilai *rating* aktual k-i
- $\hat{y}_i$ = nilai *rating* prediksi k-i

Ketiga metrik ini bernilai non-negatif dan semakin kecil nilainya semakin baik performa model. RMSE lebih sensitif terhadap kesalahan besar karena sifat pengkuadratan yang diwarisi dari MSE, sedangkan MAE memberikan gambaran kesalahan rata-rata yang lebih merata tanpa memberikan penalti berlebih pada outlier (Bobadilla et al., 2023).

2.1.7 K-Fold Cross Validation

K-fold cross validation merupakan teknik evaluasi model yang membagi seluruh data menjadi k bagian yang berukuran sama, kemudian melatih dan menguji model sebanyak k kali secara bergantian (Abedin et al., 2026). Teknik ini mengatasi keterbatasan metode *holdout* biasa yang hanya melakukan satu kali pembagian data latih dan data uji, sehingga hasil evaluasinya bergantung pada bagaimana data kebetulan terbagi. Dengan menggunakan seluruh data untuk pelatihan dan pengujian secara bergantian, *k-fold cross validation* menghasilkan estimasi performa model yang lebih stabil dan dapat direplikasi.

Prosedur *k-fold cross validation* diawali dengan mengacak seluruh data, kemudian membaginya menjadi k *fold* yang berukuran sama. Pada setiap iterasi ke- i , *fold* ke- i digunakan sebagai data uji sementara $k - 1$ *fold* sisanya digunakan sebagai data latih. Model dilatih dari awal pada setiap iterasi menggunakan data latih yang berbeda, kemudian dievaluasi pada data uji *fold* ke- i . Proses ini diulang sebanyak k kali hingga setiap *fold* telah digunakan tepat satu kali sebagai data uji.

Metrik evaluasi akhir dihitung sebagai rata-rata dari metrik yang diperoleh di setiap fold, yaitu:

$$\bar{M} = \frac{1}{k} \sum_{i=1}^k M_i \quad (21)$$

Keterangan:

- $\bar{M}$ = nilai rata-rata metrik evaluasi dari k fold
- M_i = nilai metrik evaluasi pada fold ke- i
- K = jumlah fold

Pemilihan nilai k berpengaruh pada keseimbangan antara *bias* dan *variance* dari estimasi performa. Nilai k yang terlalu kecil menghasilkan data latih yang lebih sedikit sehingga estimasi cenderung *bias*, sedangkan nilai k yang terlalu besar dapat meningkatkan *variance* antar fold (Abedin et al., 2026). Nilai $k = 10$ merupakan salah satu pilihan yang umum digunakan dalam penelitian *machine learning* (Abedin et al., 2026), meskipun pemilihan nilai k yang optimal bergantung pada karakteristik data dan model yang digunakan.

2.1.8 Uji-t Berpasangan (*Paired t-test*)

Uji-t berpasangan (*paired t-test*) merupakan salah satu metode pengujian hipotesis yang digunakan ketika data yang dibandingkan berasal dari subjek yang sama namun diukur dalam dua kondisi yang berbeda (Rahmani et al., 2025). Metode ini membandingkan rata-rata dua kelompok data yang berpasangan untuk menentukan apakah perbedaan yang teramati signifikan secara statistik atau hanya merupakan variasi acak. Dua pengukuran dikatakan berpasangan apabila setiap nilai pada kelompok pertama memiliki pasangan yang bersesuaian pada kelompok kedua dari subjek yang sama.

Uji-t berpasangan dirumuskan dengan hipotesis nol (H_0) dan hipotesis alternatif (H_1) sebagai berikut (Rahmani et al., 2025):

- $H_0: \mu_1 - \mu_2 = 0$ (tidak terdapat perbedaan signifikan antara rata-rata kedua kelompok)
- $H_1: \mu_1 - \mu_2 \neq 0$ (terdapat perbedaan signifikan antara rata-rata kedua kelompok)

Statistik uji t dihitung menggunakan selisih antar pasangan data dengan rumus (Rahmani et al., 2025):

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \quad (22)$$

Dengan derajat kebebasan

$$df = n - 1 \quad (23)$$

Keterangan:

- $\bar{d}$ = rata-rata selisih antara dua kelompok pengukuran berpasangan
- s_d = simpang baku dari selisih
- n = jumlah pasangan pengukuran
- df = derajat kebebasan

2.2 Tinjauan Empiris

2.2.1 Deep Variational Models for Collaborative Filtering-Based Recommender Systems (Bobadilla et al., 2023)

Pada penelitian ini, penulis mengembangkan sistem rekomendasi berbasis *collaborative filtering* menggunakan beberapa varian model *variational autoencoder* (VAE), termasuk VAE standar, *Bernoulli VAE*, dan *Gaussian VAE*. Tujuan penelitian ini adalah untuk mengevaluasi apakah penambahan tahap variasional pada model *autoencoder* yang sudah ada dapat meningkatkan akurasi prediksi rating dibandingkan model *deep matrix factorization* konvensional. Penulis berargumen bahwa VAE menghasilkan ruang laten yang lebih teratur dan kontinu dibandingkan *autoencoder* biasa, sehingga lebih mampu menangkap preferensi pengguna secara probabilistik. Eksperimen dilakukan menggunakan beberapa dataset *benchmark* sistem rekomendasi. Evaluasi dilakukan menggunakan metrik MAE dan RMSE dengan skema pengujian *train-test split*. Hasil penelitian menunjukkan bahwa model-model berbasis VAE secara konsisten mengungguli model *deep matrix factorization* pada seluruh dataset yang diuji. Penelitian ini menjadi landasan empiris penggunaan VAE dalam konteks *collaborative filtering* pada penelitian yang dilakukan. Namun, penelitian ini tidak mengeksplorasi penggabungan VAE dengan teknik dekomposisi matriks seperti RSVD, dan tidak menerapkan *cross-validation* dalam proses evaluasinya.

2.2.2 A Deep Autoencoder-Based Hybrid Recommender System (Bougteb et al., 2022)

Pada penelitian ini, penulis membangun sebuah sistem rekomendasi hibrid yang menggabungkan *deep autoencoder* dengan SVD++ untuk mengatasi masalah *data sparsity* dan skalabilitas pada matriks rating pengguna-item. *Deep autoencoder* dengan 5 *hidden layer* digunakan untuk mempelajari representasi laten berdimensi rendah dari matriks rating yang *sparse* dan merekonstruksi *rating* yang hilang, sementara SVD++ dijalankan secara paralel untuk mempertahankan informasi korelasi antara faktor laten pengguna dan item, termasuk *implicit feedback*. Prediksi akhir dihasilkan melalui *weighted average* dari keluaran kedua komponen tersebut, dengan bobot optimal $w = 0,9$ untuk SVD++ dan $1 - w = 0,1$ untuk *deep autoencoder*. Eksperimen dilakukan menggunakan tiga dataset, yaitu MovieLens 100K, MovieLens 1M, dan Book-Crossing. Evaluasi dilakukan menggunakan metrik RMSE dan MAE dengan skema *train-test split* 75:25. Pada dataset MovieLens 100K, hasil yang diperoleh adalah RMSE sebesar 0,992 dan MAE sebesar 0,777, mengungguli dua model *baseline* hibrid yang diuji. Penelitian ini secara langsung memotivasi pendekatan yang digunakan dalam penelitian yang dilakukan, yaitu mengombinasikan model berbasis *autoencoder* dengan teknik dekomposisi matriks melalui *weighted average*. Meskipun demikian, terdapat perbedaan mendasar yaitu penelitian Bougteb et al. menggunakan *autoencoder* deterministik dan SVD++ yang memperhitungkan *implicit feedback*, sedangkan penelitian yang dilakukan menggunakan *variational autoencoder* dengan regularisasi KL *annealing* dan RSVD dengan *bias terms* yang dioptimasi menggunakan SGD pada data *explicit feedback*. Selain itu, penelitian ini tidak menerapkan *cross-validation* dalam proses evaluasinya.

2.2.3 Implementasi Regularized Singular Value Decomposition dalam Sistem Rekomendasi Buku Collaborative Filtering (Darma Putra & Santiyasa, 2025)

Pada penelitian ini, penulis membangun sistem rekomendasi buku menggunakan metode *collaborative filtering* dengan RSVD sebagai algoritma prediksi rating. RSVD digunakan untuk mendekomposisi matriks rating pengguna-

item ke dalam representasi faktor laten dengan meminimalkan *regularized squared error* melalui SGD. Penulis juga memanfaatkan nilai faktor laten item yang dihasilkan RSVD untuk menghitung *cosine similarity* antar buku sebagai dasar rekomendasi non-personal. Data yang digunakan adalah data primer yang dikumpulkan langsung dari siswa SMP Negeri 3 Kediri, terdiri dari 886 buku, 332 pengguna, dan 124.841 data rating dengan skala 1–5. Evaluasi dilakukan menggunakan metrik MAE dan RMSE dengan skema *holdout* 80:20. Hasil terbaik yang diperoleh adalah MAE sebesar 0,478 dan RMSE sebesar 0,686 dengan kombinasi 100 faktor laten dan nilai regularisasi 0,05. Penelitian ini juga menunjukkan bahwa penambahan regularisasi secara signifikan meningkatkan akurasi prediksi dibandingkan RSVD tanpa regularisasi. Penelitian ini memperkuat justifikasi penggunaan RSVD dalam penelitian yang dilakukan, khususnya dalam hal efektivitas regularisasi terhadap akurasi prediksi. Namun, penelitian ini berfokus pada domain rekomendasi buku dengan data primer berskala kecil, tidak mengeksplorasi penggabungan RSVD dengan model *deep learning* seperti VAE, dan tidak menerapkan *cross-validation* dalam evaluasinya.

2.2.4 Hybrid Recommendation System for Movies Using Artificial Neural Network (Sharma & Shakya, 2024)

Pada penelitian ini, penulis mengembangkan sistem rekomendasi film hibrid yang mengintegrasikan empat komponen: SVD yang telah ditingkatkan untuk prediksi rating berbasis *collaborative filtering*, *Content Driven* KNN berbasis *cosine similarity* untuk deteksi kemiripan film, IKSOM berbasis *EISEN cosine correlation distance* untuk pengklusteran, dan K-Means++ dengan metode *silhouette* untuk kategorisasi data film. Prediksi akhir dihasilkan melalui kombinasi skor dari keempat komponen tersebut. Data yang digunakan adalah MovieLens 25M yang berisi 25 juta rating dari 162.541 pengguna untuk 62.423 film. Evaluasi dilakukan menggunakan metrik RMSE dan MAE dengan skema *train-test split* 75:25. Hasil yang diperoleh menunjukkan RMSE sebesar 0,414 dan MAE sebesar 0,279, mengungguli berbagai metode *baseline*. Penelitian ini menunjukkan bahwa pendekatan hibrid yang menggabungkan SVD dengan metode lain dapat meningkatkan akurasi prediksi secara signifikan pada domain rekomendasi film.

2.2.5 A Deep Learning Based Hybrid Recommendation Model for Internet Users (Sami et al., 2024)

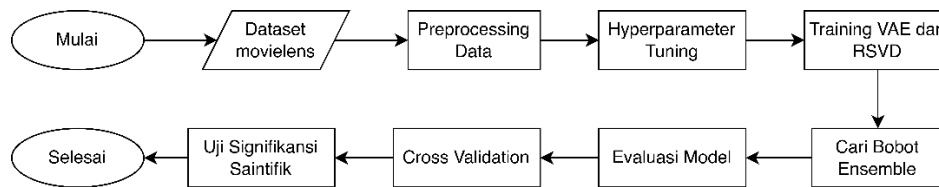
Pada penelitian ini, penulis mengembangkan model sistem rekomendasi hibrid yang mengintegrasikan *user-based* dan *item-based collaborative filtering*, *neural collaborative filtering* (NCF), *recurrent neural network* (RNN), dan *content-based filtering* berbasis TF-IDF. Prediksi akhir dihasilkan melalui kombinasi tertimbang dari keluaran seluruh komponen tersebut. Data yang digunakan adalah MovieLens 100K yang berisi 100.000 rating dari 943 pengguna untuk 1.682 film, yang merupakan dataset yang sama dengan penelitian yang dilakukan. Evaluasi dilakukan menggunakan metrik RMSE, MAE, Precision, dan Recall dengan skema *train-test split* 80:20. Hasil yang diperoleh setelah *hyperparameter tuning* adalah RMSE sebesar 0,7723 dan MAE sebesar 0,6018, mengungguli berbagai model *baseline* yang diuji pada dataset yang sama.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Desain Sistem

Penelitian ini mengembangkan sistem rekomendasi film berbasis prediksi rating menggunakan pendekatan hybrid ensemble yang menggabungkan model Variational Autoencoder (VAE) dan Regularized Singular Value Decomposition (RSVD). Prediksi dari kedua model digabungkan melalui mekanisme weighted average ensemble untuk menghasilkan prediksi rating akhir. Secara keseluruhan, penelitian ini mencakup delapan tahap utama sebagaimana diilustrasikan pada **Gambar 3.1.**



Gambar 3.1. Metode Penelitian

Penelitian dimulai dengan pengumpulan dataset MovieLens 100K, dilanjutkan dengan *preprocessing* data yang mencakup normalisasi rating dan pembagian data menjadi partisi latih, validasi, dan uji. Selanjutnya dilakukan *hyperparameter tuning* untuk memperoleh konfigurasi optimal bagi kedua model, yang kemudian digunakan pada tahap pelatihan. Bobot *ensemble* dicari pada data validasi dan diterapkan saat evaluasi pada data uji. Untuk mengukur stabilitas dan generalisasi model secara lebih komprehensif, evaluasi dilanjutkan dengan *10-fold cross-validation* pada keseluruhan dataset, diikuti uji signifikansi statistik menggunakan *paired t-test*.

3.2 Dataset

Data yang digunakan dalam penelitian ini adalah dataset MovieLens 100K yang diperoleh secara sekunder dari situs *grouplens.org*. Dataset ini merupakan salah satu dataset *benchmark* yang umum digunakan dalam penelitian sistem rekomendasi (Sami et al., 2024). MovieLens 100K terdiri dari 100.000 data rating yang diberikan oleh 943 pengguna terhadap 1.682 film, dengan skala rating 1

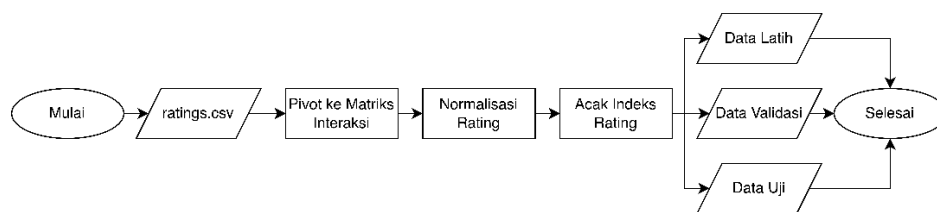
hingga 5. File yang digunakan dalam penelitian ini adalah *ratings.csv* yang memuat kolom *userId*, *movieId*, *rating*, dan *timestamp* sebagaimana ditunjukkan pada **Tabel 3.1**. Kolom *timestamp* tidak digunakan dalam penelitian ini karena model yang dikembangkan tidak mempertimbangkan dimensi waktu dalam prediksi rating.

Tabel 3.1. Contoh Data *ratings.csv*

userId	movieId	rating	timestamp
1	1	4.0	964982703
1	3	4.0	964981247
1	6	4.0	964982224
1	47	5.0	964983815
1	50	5.0	964982931

3.3 Preprocessing Data

Preprocessing data dilakukan dalam tiga tahap utama, yaitu pembangunan matriks interaksi pengguna-film, normalisasi rating, dan pembagian data. Alur keseluruhan tahap *preprocessing* diilustrasikan pada **Gambar 3.2**.



Gambar 3.2. Metode Penelitian

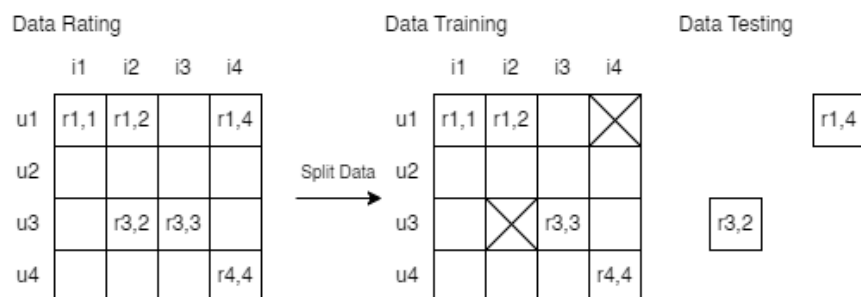
Tahap pertama adalah pembangunan matriks interaksi pengguna-film. Data pada *ratings.csv* dipivot menjadi matriks berukuran 943×1.682 , di mana baris merepresentasikan pengguna dan kolom merepresentasikan film. Setiap sel pada matriks berisi nilai rating yang diberikan pengguna terhadap film yang bersangkutan, sedangkan sel yang tidak memiliki rating diisi dengan nilai 0. Contoh matriks interaksi ditunjukkan pada **Tabel 3.2** dimana indeks baris adalah pengguna dan kolom adalah *film*.

Tabel 3.2. Contoh Matriks Interaksi

	1	2	3	4	5	6	...
1	4.0	0.0	4.0	0.0	0.0	4.0	...
2	0.0	0.0	0.0	0.0	0.0	0.0	...
3	0.0	0.0	0.0	0.0	0.0	0.0	...
4	0.0	0.0	0.0	0.0	0.0	0.0	...
...	...	...	...	...	...	...	...

Tahap kedua adalah normalisasi rating. Seluruh nilai rating pada matriks dinormalisasi ke rentang $[0, 1]$ dengan membagi setiap nilai dengan nilai rating maksimum (5.0). Normalisasi ini diperlukan agar nilai masukan sesuai dengan fungsi aktivasi *sigmoid* pada lapisan *output decoder* VAE yang menghasilkan keluaran dalam rentang yang sama. Sel kosong yang bernilai 0 tetap bernilai 0 setelah normalisasi sehingga dapat dibedakan dari rating terendah yang bernilai 0.2.

Tahap ketiga adalah pembagian data menggunakan metode *hold-out* pada level rating individual. Seluruh 100.000 rating diacak dengan *random seed* 42 untuk memastikan reproduksibilitas, kemudian dibagi menjadi tiga partisi: 70% data latih (70.000 rating) untuk melatih VAE dan RSVD, 10% data validasi (10.000 rating) untuk menentukan bobot *ensemble* optimal, dan 20% data uji (20.000 rating) untuk evaluasi performa akhir model. Pembagian dilakukan pada level rating individual, bukan pada level pengguna, sehingga setiap pengguna dapat muncul di ketiga partisi sekaligus. Ilustrasi pembagian data ditunjukkan pada Gambar 3.3.



Gambar 3.3. Ilustrasi Pembagian Data

3.4 Perancangan Model

3.4.1 Variational Autoencoder (VAE)

VAE yang dikembangkan dalam penelitian ini terdiri dari tiga komponen utama: *encoder*, *sampling*, dan *decoder*. *Encoder* menerima masukan berupa vektor rating pengguna dari matriks interaksi berukuran 1.682 dimensi. Sebelum melewati lapisan tersembunyi, masukan dikenai *dropout* untuk mencegah *overfitting*. Lapisan tersembunyi *encoder* terdiri dari dua lapisan *dense* dengan fungsi aktivasi ReLU, diikuti dua lapisan keluaran yang masing-masing menghasilkan vektor μ (z_mean) dan $\log \sigma^2$ (z_log_var) yang merepresentasikan rata-rata dan log varians distribusi ruang laten.

Komponen *sampling* mengimplementasikan *reparameterization trick* agar proses pengambilan sampel dari ruang laten tetap dapat diturunkan secara matematis (*differentiable*) sehingga *backpropagation* dapat berjalan. Titik sampel laten z diperoleh melalui persamaan (7).

Decoder menerima vektor laten z dan merekonstruksinya kembali menjadi vektor prediksi rating berukuran 1.682 dimensi. Lapisan tersembunyi *decoder* merupakan kebalikan simetris dari *encoder*, yaitu dua lapisan *dense* dengan fungsi aktivasi ReLU. Lapisan keluaran *decoder* menggunakan fungsi aktivasi *sigmoid* yang menghasilkan nilai dalam rentang $[0, 1]$, sesuai dengan skala rating yang telah dinormalisasi.

Fungsi *loss* yang digunakan adalah fungsi *loss* Beta-VAE yang merupakan kombinasi dari *reconstruction loss* dan *KL divergence loss* menggunakan persamaan (6). *Reconstruction loss* dihitung menggunakan *masked MSE* yang hanya mempertimbangkan sel matriks yang memiliki nilai rating, sehingga model tidak belajar dari nilai 0 yang merepresentasikan rating yang tidak ada menggunakan persamaan (5). *KL divergence loss* meregulasi distribusi ruang laten agar mendekati distribusi normal standar menggunakan persamaan (4).

Hyperparameter β dikontrol secara dinamis melalui mekanisme *KL annealing* untuk mencegah *posterior collapse*, yaitu kondisi di mana *encoder* mengabaikan masukan dan menghasilkan distribusi laten yang identik untuk semua

pengguna (Wang et al., 2025). Pada awal pelatihan β ditetapkan sebesar 0 sehingga model fokus meminimalkan *reconstruction loss* terlebih dahulu. Nilai β kemudian dinaikkan secara linear hingga mencapai nilai target yang diperoleh dari hasil *hyperparameter tuning*. Model dioptimasi menggunakan algoritma Adam. Nilai *learning rate*, *batch size*, *dropout rate*, dimensi ruang laten, dan jumlah unit pada setiap lapisan tersembunyi ditentukan melalui proses *hyperparameter tuning*.

3.4.2 Regularized Singular Value Decomposition (RSVD)

RSVD yang dikembangkan dalam penelitian ini mengimplementasikan faktorisasi matriks dengan tiga komponen utama yaitu bias global (μ), bias pengguna (b_u), dan bias item (b_i), di samping matriks faktor laten U , Σ , dan V . Prediksi rating pengguna u terhadap item i diformulasikan dengan persamaan (9). Parameter model diinisialisasi sebagai berikut:

- μ dihitung sebagai rata-rata global dari seluruh rating pada data latih
- b_u dan b_i diinisialisasi dengan nilai 0
- matriks U dan V diinisialisasi dengan nilai acak dari distribusi normal dengan skala 0.1
- matriks Σ diinisialisasi sebagai matriks identitas.

Inisialisasi Σ sebagai matriks identitas dipilih untuk mencegah *vanishing gradient* pada epoch-epoch awal pelatihan, karena inisialisasi dengan nilai mendekati 0 menyebabkan gradien U dan V ikut terhambat secara kaskade.

Model dilatih menggunakan *Stochastic Gradient Descent* (SGD) dengan fungsi *loss* yang merupakan kombinasi *reconstruction loss* dan penalti regularisasi L2 menggunakan persamaan (10). Pembaruan parameter dilakukan hanya pada pasangan (u, i) yang memiliki rating aktual, sehingga sel kosong pada matriks interaksi tidak mempengaruhi proses pelatihan. Aturan pembaruan untuk setiap parameter adalah dilakukan melalui persamaan (12) – (15). Perlu diperhatikan bahwa regularisasi L2 diterapkan pada U , V , b_u , dan b_i , namun tidak diterapkan pada Σ . Keputusan desain ini diambil berdasarkan eksperimen awal yang menunjukkan bahwa regularisasi pada Σ menyebabkan nilai-nilainya mendekati 0 secara kaskade sehingga menghambat gradien U dan V .

Untuk mencegah *overfitting*, RSVD menggunakan mekanisme *early stopping* berbasis *validation MSE*. Sebelum pelatihan dimulai, sebanyak 10% dari

rating data latih disisihkan sebagai *validation set* internal menggunakan metode *hold-out*. Pada setiap akhir epoch, model mengevaluasi *validation MSE* dan menyimpan *snapshot* bobot terbaik. Pelatihan dihentikan apabila tidak terdapat perbaikan *validation MSE* selama sejumlah epoch yang ditentukan oleh parameter *patience*. Setelah pelatihan selesai, bobot dikembalikan ke *snapshot* terbaik. *Validation MSE* digunakan sebagai kriteria *early stopping* karena *training MSE* selalu menurun secara monoton sehingga tidak mencerminkan performa model pada data yang belum dilihat. Nilai *learning rate*, koefisien regularisasi λ , jumlah faktor laten, dan *patience* ditentukan melalui proses *hyperparameter tuning*.

3.4.3 Model Hybrid dengan Weighted Average Ensemble

Penelitian ini mengembangkan model *hybrid* yang menggabungkan prediksi VAE dan RSVD melalui mekanisme *weighted average ensemble*. Pendekatan *hybrid* dipilih karena VAE dan RSVD memodelkan preferensi pengguna dari sudut pandang yang berbeda, VAE mempelajari representasi laten melalui distribusi probabilitas, sedangkan RSVD mengdekomposisi matriks rating melalui faktorisasi laten deterministik dengan bias terms.

Sebelum digabungkan, prediksi dari kedua model dinormalisasi ke skala rating asli (1–5) dan dibatasi (*clipping*) agar tidak melampaui rentang tersebut. Prediksi *hybrid* diformulasikan dengan persamaan (16) dan (17). Nilai α optimal ditentukan melalui *grid search* pada data validasi (10%) yang telah disisihkan pada tahap *preprocessing*. Pencarian dilakukan dengan mencoba seluruh nilai α dari 0.00 hingga 1.00 dengan langkah 0.01, menghasilkan 101 kandidat nilai. Untuk setiap nilai α , RMSE dihitung antara prediksi *hybrid* dan rating aktual pada data validasi. Nilai α yang menghasilkan RMSE terendah di data validasi dipilih sebagai bobot final dan digunakan untuk menghitung prediksi *hybrid* pada data uji. Dengan pendekatan ini, pemilihan α sepenuhnya didasarkan pada data validasi sehingga evaluasi pada data uji tetap tidak bias.

3.5 Hyperparameter Tuning

Hyperparameter tuning dilakukan secara terpisah untuk VAE dan RSVD menggunakan data latih (70%). Tujuan dari tahap ini adalah menemukan

konfigurasi hyperparameter yang menghasilkan performa terbaik sebelum model dilatih secara final dan dievaluasi.

3.5.1 Hyperparameter Tuning VAE

Tuning VAE dilakukan menggunakan *framework* Optuna dengan algoritma *Tree-structured Parzen Estimator* (TPE). TPE membangun model probabilistik dari hasil *trial* sebelumnya untuk menentukan kombinasi hyperparameter berikutnya yang paling menjanjikan agar pencarian menjadi lebih efisien. Setiap *trial* melatih VAE selama 30 *epoch* dan menggunakan *reconstruction loss* pada epoch terakhir sebagai kriteria seleksi. *Reconstruction loss* dipilih sebagai kriteria bukan *total loss* agar hasil tidak bias terhadap nilai β yang kecil, karena *total loss* yang rendah dapat dicapai sekadar dengan meminimalkan kontribusi KL divergence melalui β kecil tanpa benar-benar meningkatkan kualitas rekonstruksi. Total *trial* yang dijalankan adalah 200. Ruang pencarian hyperparameter VAE ditunjukkan pada Tabel 3.3.

Tabel 3.3. Ruang Pencarian *Hyperparameter* VAE

<i>Hyperparameter</i>	Nama Variabel	Ruang Pencarian
Dimensi ruang laten	latent_dim	20, 50, 100, 200
Kecepatan belajar	learning_rate	0.0001, 0.0005, 0.001, 0.005, 0.01
Jumlah sampel per iterasi	batch_size	64, 128, 256
Laju <i>dropout</i> pada lapisan tersembunyi	dropout_rate	0.1, 0.2, 0.3, 0.4
Jumlah unit pada setiap lapisan tersembunyi	hidden_dims	[512, 256], [512, 256, 128], [1024, 512], [1024, 512, 256]
Bobot KL divergence pada fungsi <i>loss</i> Beta-VAE	beta	0.01, 0.05, 0.1, 0.3, 0.5, 0.8, 1.0

Pemilihan rentang kandidat untuk setiap hyperparameter VAE didasarkan pada karakteristik dataset dan arsitektur model.

- `latent_dim` divariasikan dari 20 hingga 200 untuk mengeksplorasi trade-off antara kapasitas representasi ruang laten dan risiko *overfitting*. Dimensi terlalu kecil tidak mampu menangkap kompleksitas preferensi pengguna, sedangkan dimensi terlalu besar berpotensi mempelajari *noise*.
- `learning_rate` mencakup rentang lebar dari 0.0001 hingga 0.01 karena sensitivitas konvergensi VAE terhadap nilai ini cukup tinggi, terutama akibat interaksi antara *reconstruction loss* dan KL divergence.
- `batch_size` dibatasi pada nilai 64, 128, dan 256 mengingat ukuran dataset (943 pengguna). *Batch size* yang terlalu besar akan menghasilkan terlalu sedikit iterasi per epoch.
- `dropout_rate` diuji pada rentang 0.1 - 0.4 karena nilai di bawah 0.1 memberikan regularisasi yang tidak signifikan, sedangkan nilai di atas 0.4 berisiko menghilangkan terlalu banyak informasi dari vektor rating yang sudah sangat *sparse*.
- `hidden_dims` mencakup empat variasi arsitektur dua dan tiga lapisan dengan jumlah unit yang berbeda untuk mengeksplorasi kedalaman dan lebar jaringan secara bersamaan.
- `beta` mencakup rentang 0.01 hingga 1.0 karena nilai β yang terlalu besar cenderung menyebabkan *posterior collapse*, sedangkan nilai yang terlalu kecil membuat regularisasi ruang laten tidak efektif.

3.5.2 Hyperparameter Tuning RSVD

Tuning RSVD dilakukan menggunakan *exhaustive grid search* yang mencoba seluruh kombinasi dari ruang pencarian secara sistematis. Total kombinasi yang dievaluasi adalah $3 \times 4 \times 4 = 48$ kombinasi. Setiap kombinasi melatih RSVD selama 15 *epoch* dan menggunakan *total loss* (MSE + penalti L2) pada epoch terakhir sebagai kriteria seleksi. Ruang pencarian hyperparameter RSVD ditunjukkan pada Tabel 3.4.

Tabel 3.4. Ruang Pencarian *Hyperparameter* RSVD

<i>Hyperparameter</i>	Nama Variabel	Ruang Pencarian
Jumlah dimensi faktor laten	<code>n_factors</code>	10, 20, 50
Kecepatan belajar SGD	<code>learning_rate</code>	0.001, 0.005, 0.01, 0.02
Koefisien penalti regularisasi L2	<code>lambda_reg</code>	0.001, 0.005, 0.01, 0.02

Pemilihan rentang kandidat untuk setiap hyperparameter RSVD mempertimbangkan karakteristik dataset dan kompleksitas komputasi.

- `n_factors` diuji pada nilai 10, 20, dan 50 karena RSVD dengan jumlah faktor laten yang besar membutuhkan waktu komputasi yang jauh lebih lama akibat iterasi SGD per pasangan (u, i).
- `learning_rate` diuji pada rentang 0.001 hingga 0.02 karena SGD pada RSVD lebih sensitif terhadap nilai ini dibandingkan optimizer Adam pada VAE. Nilai terlalu besar menyebabkan divergensi, sedangkan nilai terlalu kecil memperlambat konvergensi secara signifikan.
- `lambda_reg` diuji pada rentang 0.001 hingga 0.02 untuk menyeimbangkan antara regularisasi yang cukup untuk mencegah *overfitting* dan tidak terlalu agresif sehingga menghambat kemampuan model mempelajari pola rating.

3.6 Evaluasi Model

Setelah hyperparameter optimal diperoleh melalui proses *hyperparameter tuning*, VAE dan RSVD dilatih secara final menggunakan data latih (70%). Bobot *ensemble α* kemudian ditentukan pada data validasi (10%) melalui prosedur *grid search*. Model yang telah dilatih selanjutnya dievaluasi melalui dua tahap. Tahap pertama adalah evaluasi *hold-out* pada data uji (20%) yang tidak pernah digunakan pada proses pelatihan maupun penentuan bobot *ensemble*. Tahap kedua adalah *10-fold cross-validation* pada keseluruhan dataset untuk mengukur stabilitas dan kemampuan generalisasi model secara lebih komprehensif. Signifikansi statistik perbedaan performa antar model kemudian diuji menggunakan *paired t-test*.

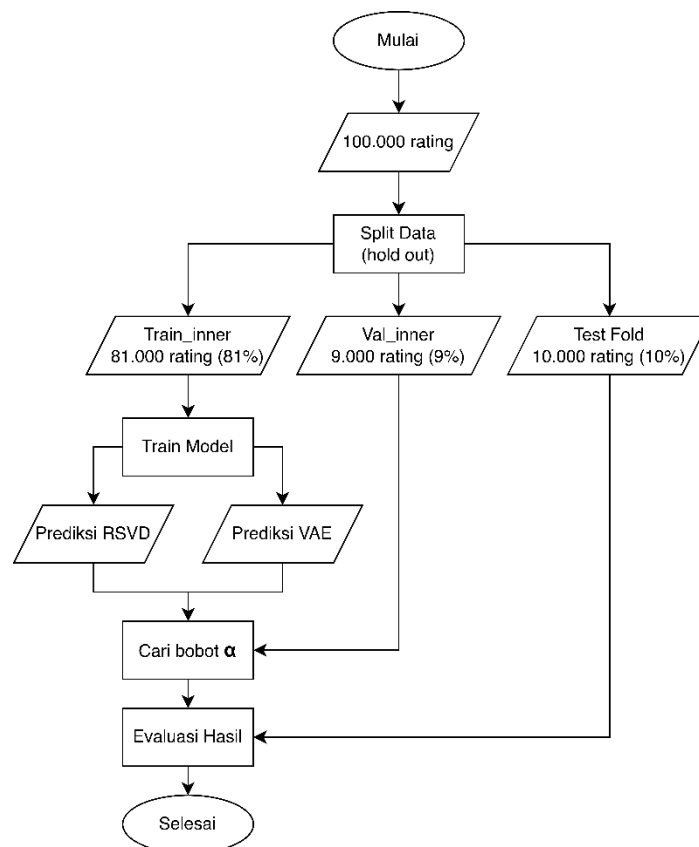
3.6.1 Metrik Evaluasi

Evaluasi performa model diukur menggunakan dua metrik, yaitu *Mean Absolute Error* (MAE) dan *Root Mean Squared Error* (RMSE) sebagaimana diformulasikan pada persamaan (18) dan (20). Kedua metrik ini mengukur selisih antara nilai rating aktual dengan nilai rating yang diprediksi oleh model. MAE memberikan gambaran rata-rata kesalahan prediksi secara merata, sedangkan RMSE memberikan penalti yang lebih besar terhadap kesalahan prediksi yang jauh dari nilai aktual karena sifat pengkuadratnya. Nilai kedua metrik bersifat non-negatif dan semakin kecil nilainya semakin baik performa model.

3.6.2 10-Fold Cross-Validation

10-fold cross-validation dilakukan pada keseluruhan dataset untuk mengukur stabilitas dan kemampuan generalisasi model secara lebih komprehensif dibandingkan evaluasi hold-out tunggal. Nilai $k = 10$ dipilih berdasarkan

pertimbangan keseimbangan antara bias dan variance dari estimasi performa sebagaimana dijelaskan pada tinjauan teori.



Gambar 3.4. Struktur *Cross Validation*

Pembagian fold dilakukan pada level rating individual, bukan pada level pengguna. Seluruh rating dikelompokkan ke dalam 10 fold secara acak, sehingga setiap fold berisi sekitar 10% dari total rating. Struktur partisi data pada setiap fold diilustrasikan pada **Gambar 3.4**. Pada setiap iterasi fold ke- i , sekitar 10.000 *rating* pada fold ke- i digunakan sebagai data uji (*test fold*), sedangkan 90.000 *rating* sisanya dibagi lebih lanjut menjadi data latih-dalam (*train_inner*, ± 81.000 *rating*) dan data validasi-dalam (*val_inner*, ± 9.000 *rating*) menggunakan skema *hold-out* 90/10.

Dari masing-masing partisi ini, dibangun dua matriks interaksi pengguna-item yang terpisah, matriks *train_inner* yang hanya berisi rating pada posisi *train_inner*, dan matriks uji yang hanya berisi rating pada posisi *test fold*. Posisi

yang tidak termasuk dalam partisi masing-masing matriks dibiarkan bernilai nol dan dikecualikan dari perhitungan metrik melalui masked loss pada VAE dan seleksi indeks eksplisit pada RSVD.

Pada setiap iterasi, VAE dan RSVD dilatih secara paralel dari awal menggunakan `train_inner` dengan hyperparameter optimal yang diperoleh dari Sub-bab 3.5. Prediksi dari kedua model kemudian dihasilkan pada posisi `rating_val_inner` untuk keperluan pencarian bobot ensemble. Pencarian bobot α dilakukan melalui grid search pada `val_inner` menggunakan prosedur yang sama dengan Sub-bab 3.4.3. Bobot α terbaik yang diperoleh dari `val_inner` selanjutnya digunakan untuk menghitung prediksi hybrid pada test fold, di mana RMSE dan MAE dihitung untuk ketiga model (VAE, RSVD, dan hybrid) menggunakan rating aktual *test fold*. Hasil setiap *fold* disimpan dan proses ini diulang sebanyak 10 kali hingga seluruh fold telah digunakan sebagai data uji tepat satu kali. Metrik akhir dilaporkan sebagai rata-rata $\pm$ standar deviasi dari 10 *fold* menggunakan [persamaan \(21\)](#).

3.6.3 Uji Signifikansi Statistik (*Paired t-Test*)

Uji signifikansi statistik dilakukan untuk membuktikan bahwa perbedaan performa adalah perbedaan yang bermakna secara statistik. Pengujian *paired t-test* menggunakan [persamaan \(22\) dan \(23\)](#), dengan data berpasangan berupa metrik per *fold* dari *10-fold cross-validation*. Setiap pasangan nilai metrik dari dua model yang dibandingkan diperlakukan sebagai pengamatan berpasangan karena keduanya dievaluasi pada partisi data uji yang identik di setiap *fold*.

Pengujian dilakukan pada metrik RMSE dan MAE, mencakup tiga pasangan perbandingan yaitu *Hybrid* vs. RSVD, *Hybrid* vs. VAE, dan RSVD vs. VAE. Hipotesis yang diuji adalah sebagai berikut:

- H_0 : Tidak terdapat perbedaan yang signifikan antara rata-rata metrik kedua model yang dibandingkan
- H_1 : Terdapat perbedaan yang signifikan antara rata-rata metrik kedua model yang dibandingkan

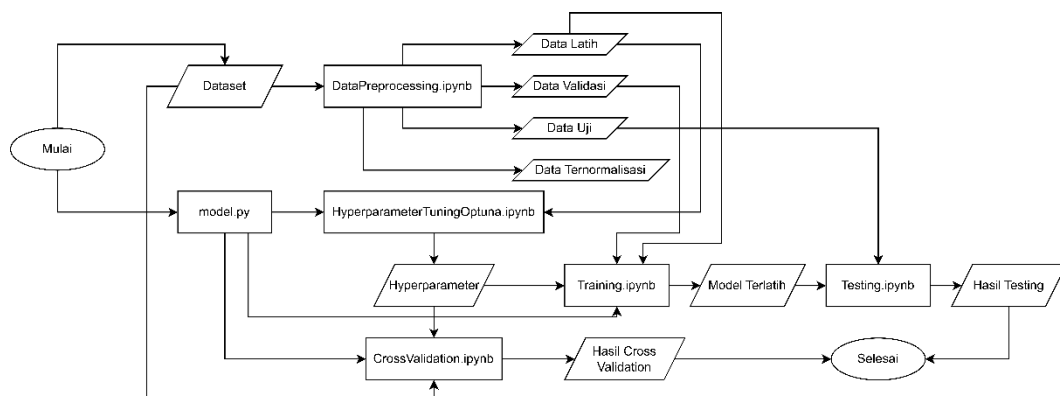
Tingkat signifikansi yang digunakan adalah $\alpha = 0,05$. H_0 ditolak apabila *p-value* $< 0,05$, yang berarti perbedaan performa antar model signifikan secara statistik pada tingkat kepercayaan 95%.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

Sistem rekomendasi film yang dikembangkan dalam penelitian ini diimplementasikan menggunakan bahasa pemrograman Python dengan memanfaatkan beberapa *library* utama, yaitu NumPy dan Pandas untuk manipulasi data, TensorFlow dan Keras untuk pembangunan dan pelatihan model *Variational Autoencoder* (VAE), serta SciPy untuk pengujian statistik. Implementasi sistem diorganisasi ke dalam enam komponen utama yang saling terhubung, yaitu satu *script* definisi model (*model.py*) dan lima *notebook* Jupyter yang masing-masing menangani tahapan pipeline yang berbeda. Hubungan antar komponen tersebut diilustrasikan pada **Gambar 4.1**.



Gambar 4.1. Hubungan Antar Komponen

Gambar 4.1 menunjukkan bahwa pipeline sistem dimulai dari Dataset MovieLens 100K yang diproses oleh *DataPreprocessing.ipynb* menghasilkan empat keluaran yaitu Data Latih, Data Validasi, Data Uji, dan Data Ternormalisasi. *HyperParameterTuningOptuna.ipynb* mengonsumsi *model.py* dan Data Latih untuk mencari konfigurasi hyperparameter optimal, yang hasilnya disimpan sebagai Hyperparameter dan digunakan bersama oleh *Training.ipynb* dan *CrossValidation.ipynb*. *Training.ipynb* menerima Data Latih, Data Validasi, dan Hyperparameter, kemudian menghasilkan Model Terlatih yang menjadi masukan bagi *Testing.ipynb*. *CrossValidation.ipynb* menerima Data Ternormalisasi,

Hyperparameter, dan definisi kelas dari model.py untuk menjalankan evaluasi 10-*fold* secara independen dan menghasilkan Hasil Cross Validation. Testing.ipynb menggunakan Model Terlatih dan Data Uji untuk menghasilkan Hasil Testing. Deskripsi masing-masing komponen beserta fungsinya dirangkum pada **Tabel 4.1**.

Tabel 4.1. Deskripsi Komponen Implementasi Sistem

Komponen	Fungsi
model.py	Mendefinisikan seluruh kelas model (Sampling, Encoder, Decoder, VAE, KLANnealingCallback, dan RSVD)
DataPreprocessing.ipynb	Memuat dataset MovieLens 100K, membangun matriks interaksi <i>user-item</i> , melakukan normalisasi rating, dan membagi data menjadi <i>train/validation/test set</i>
HyperParameterTuningOptuna.ipynb	Melakukan pencarian hyperparameter optimal VAE menggunakan Optuna TPE (200 <i>trial</i>) dan hyperparameter RSVD menggunakan <i>grid search</i> (48 kombinasi)
Training.ipynb	Melatih VAE dan RSVD menggunakan hyperparameter terbaik hasil tuning dalam satu sesi penuh, menyimpan bobot model, dan memvisualisasikan kurva pelatihan
CrossValidation.ipynb	Menjalankan evaluasi 10- <i>fold cross-validation</i> , menghitung metrik per <i>fold</i> , merangkum statistik, dan melakukan <i>paired t-test</i> antar model
Testing.ipynb	Memuat bobot model hasil Training.ipynb, menjalankan prediksi

	pada <i>test set</i> final, mencari bobot ensemble optimal pada <i>validation set</i> , dan mengevaluasi performa ketiga model
--	--

4.1.1 Program Definisi Model

File `model.py` merupakan komponen inti yang mendefinisikan seluruh arsitektur model dalam penelitian ini. File ini terdiri dari enam kelas yang diorganisasi berdasarkan tanggung jawabnya masing-masing. Potongan kode yang mendefinisikan kelas-kelas tersebut ada pada **Tabel 4.2 hingga Tabel 4.7**.

Tabel 4.2. Kelas *Sampling*

```
class Sampling(Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]

        # Mengambil noise acak (epsilon) dari distribusi normal
        standar (mean=0, std=1)
        epsilon = tf.keras.backend.random_normal(shape=(batch,
dim))

        # Menghitung standar deviasi (sigma) dari log_var
        sigma = tf.exp(0.5 * z_log_var)

        # Mengembalikan titik sampel laten: z = mean + (sigma *
epsilon)
        return z_mean + (sigma * epsilon)
```

Tabel 4.2 adalah potongan program untuk kelas *sampling*. Kelas *Sampling* mengimplementasikan *reparameterization trick* yang memungkinkan operasi pengambilan sampel acak dari ruang laten tetap dapat diturunkan secara matematis (*differentiable*). Tanpa teknik ini, gradien tidak dapat mengalir melewati operasi *sampling* sehingga *backpropagation* tidak dapat berjalan. Kelas ini mewarisi *Layer* dari Keras dan hanya mendefinisikan satu *method* yaitu *call*, yang menerima

pasangan z_mean dan z_log_var dari *encoder* lalu menghasilkan titik sampel z menggunakan **persamaan (7)**.

Tabel 4.3. Kelas *Encoder*

```
class Encoder(Model):
    def __init__(self, hidden_dims=[512, 256], latent_dim=50,
dropout_rate=0.5, **kwargs):
        super(Encoder, self).__init__(**kwargs)

        # Dropout digunakan untuk mencegah overfitting dengan
mematikan sebagian neuron secara acak
        self.dropout = Dropout(dropout_rate)

        # Membangun lapisan tersembunyi (Hidden Layers) secara
dinamis
        self.hidden_layers = [Dense(dim, activation='relu') for
dim in hidden_dims]

        # Output layer dari Encoder: Nilai Rata-rata (Mean) dan
Log Varians dari distribusi ruang laten
        self.z_mean = Dense(latent_dim, name='z_mean')
        self.z_log_var = Dense(latent_dim, name='z_log_var')

    def call(self, inputs):
        x = self.dropout(inputs)
        for layer in self.hidden_layers:
            x = layer(x)
        return self.z_mean(x), self.z_log_var(x)
```

Tabel 4.3 adalah potongan program untuk kelas *encoder*. Kelas *Encoder* memetakan vektor rating pengguna berdimensi tinggi menjadi representasi laten berdimensi rendah. Arsitekturnya dibangun secara dinamis berdasarkan parameter `hidden_dims`, sehingga jumlah dan ukuran lapisan tersembunyi dapat dikonfigurasi tanpa mengubah kode. Lapisan *dropout* diterapkan pada *input* untuk mencegah *overfitting*. *Encoder* menghasilkan dua keluaran yaitu z_mean dan z_log_var yang

merupakan parameter distribusi Gaussian dari ruang laten, bukan titik tunggal, sesuai dengan prinsip probabilitas VAE.

Tabel 4.4. Kelas *Decoder*

```
class Decoder(Model):
    def __init__(self, hidden_dims=[256, 512], output_dim=1682,
**kwargs):
        super(Decoder, self).__init__(**kwargs)
        self.hidden_layers = [Dense(dim, activation='relu') for
dim in hidden_dims]

        # Layer output menggunakan Sigmoid karena data input
sudah dinormalisasi ke rentang 0 - 1
        self.reconstruction = Dense(output_dim,
activation='sigmoid', name='decoder_output')

    def call(self, inputs):
        x = inputs
        for layer in self.hidden_layers:
            x = layer(x)
        return self.reconstruction(x)
```

Tabel 4.4 adalah potongan program untuk kelas *decoder*. Kelas *Decoder* memiliki struktur simetris dengan *encoder* dimana lapisan tersembunyinya dibangun secara dinamis dari parameter `hidden_dims` yang diurutkan terbalik. *Layer* keluaran menggunakan fungsi aktivasi *sigmoid* yang membatasi nilai rekonstruksi ke rentang $[0, 1]$, sesuai dengan skala data yang telah dinormalisasi. Parameter `output_dim` diset sesuai jumlah film unik dalam dataset (1.682 film pada MovieLens 100K).

Tabel 4.5. Kelas VAE

```
class VAE(Model):
    # Tambahkan parameter beta di init (default 1.0 seperti VAE
biasa)
    def __init__(self, encoder, decoder, beta=1.0, **kwargs):
```

```

        super(VAE, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

        # Simpan beta sebagai tf.Variable agar nilainya bisa
        diperbarui secara
        # dinamis oleh KLANnealingCallback di setiap epoch tanpa
        perlu rebuild model
        self.beta = tf.Variable(float(beta), trainable=False,
                                dtype=tf.float32, name='beta')

        self.sampling = Sampling()

        # Pelacak nilai metrik selama proses training
        self.total_loss_tracker =
tf.keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker =
tf.keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker =
tf.keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [self.total_loss_tracker,
self.reconstruction_loss_tracker, self.kl_loss_tracker]

    def call(self, inputs):
        # Forward pass biasa untuk tahap evaluasi/testing
        z_mean, z_log_var = self.encoder(inputs)
        z = self.sampling([z_mean, z_log_var])
        return self.decoder(z)

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0] # ambil x saja, y dibuang

        with tf.GradientTape() as tape:

```

```

        z_mean, z_log_var = self.encoder(data)
        z = self.sampling([z_mean, z_log_var])
        reconstruction = self.decoder(z)

        # A. Reconstruction Loss (Masked MSE)
        # Membuat mask bernilai 1 jika rating ada, dan 0
        jika kosong
        mask = tf.cast(data > 0, tf.float32)
        squared_diff = tf.square(data - reconstruction)
        masked_se = squared_diff * mask

        # Menghitung rata-rata error murni
        mse_loss = tf.reduce_sum(masked_se) /
        (tf.reduce_sum(mask) + 1e-8)
        num_items = tf.cast(tf.shape(data)[1], tf.float32)
        reconstruction_loss = mse_loss * num_items

        # B. KL Divergence Loss
        # Berfungsi meregulasi agar distribusi ruang laten
        mendekati normal Gaussian
        kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean)
        - tf.exp(z_log_var))
        kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss,
        axis=1))

        # C. Total Loss dengan Hyperparameter Beta
        # Gunakan self.beta yang didapat dari inisialisasi
        model
        total_loss = reconstruction_loss + (self.beta *
        kl_loss)

        # Backpropagation: Menghitung gradien dan memperbarui
        bobot (weights)
        grads = tape.gradient(total_loss,
        self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads,
        self.trainable_weights))

```

```

        # Memperbarui history loss untuk ditampilkan di progress
bar
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruc
tion_loss)
        self.kl_loss_tracker.update_state(kl_loss)

        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss":
self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result(),
        }

```

Tabel 4.5 adalah potongan program untuk kelas VAE. Kelas VAE menggabungkan Encoder, Sampling, dan Decoder menjadi satu model yang dapat dilatih. Kelas ini memiliki empat *method* utama:

- **`__init__`** menginisialisasi komponen model dan menetapkan beta sebagai `tf.Variable` karena dua alasan. Pertama, `tf.Variable` bersifat *mutable* sehingga nilainya dapat diubah secara *in-place* melalui `.assign()` tanpa membuat tensor baru, memungkinkan `KLAnnealingCallback` memperbarui beta di setiap *epoch* tanpa perlu membangun ulang model. Kedua, perubahan nilai tersebut terjadi di dalam *computational graph* TensorFlow sehingga kompatibel dengan mekanisme *graph execution* dan tidak memicu *retracing* pada setiap pembaruan. Tiga pelacak metrik (`total_loss_tracker`, `reconstruction_loss_tracker`, `kl_loss_tracker`) juga diinisialisasi di sini untuk memantau perkembangan *loss* selama pelatihan.
- **`metrics`** (*property*) mengembalikan daftar pelacak metrik tersebut agar Keras dapat *reset* nilainya secara otomatis di awal setiap *epoch*.
- **`call`** mendefinisikan *forward pass* standar yang digunakan saat evaluasi dan inferensi.
- **`train_step`** mengandung *masked reconstruction loss* dimana model hanya menghitung *error* pada posisi rating yang memiliki nilai eksplisit (> 0) dengan membuat *mask* biner, sehingga sel-sel kosong yang merepresentasikan film yang belum ditonton tidak ikut mempengaruhi gradien. *Total loss* dihitung sebagai jumlah *reconstruction loss* dan *KL divergence loss* yang dikali beta, sesuai dengan **persamaan (6)**.

Tabel 4.6. Kelas *KLAnnealingCallback*

```
class KLANnealingCallback(tf.keras.callbacks.Callback):
    def __init__(self, beta_target, annealing_epochs):
        super(KLANnealingCallback, self).__init__()
        self.beta_target = beta_target
        self.annealing_epochs = annealing_epochs

    def on_epoch_begin(self, epoch, logs=None):
        # Hitung proporsi kemajuan annealing (0.0 hingga 1.0)
        if self.annealing_epochs > 0:
            progress = min(epoch / self.annealing_epochs, 1.0)
        else:
            progress = 1.0

        # Hitung nilai beta saat ini secara linear
        beta_now = progress * self.beta_target

        # Perbarui tf.Variable beta di model tanpa perlu rebuild
        self.model.beta.assign(beta_now)

    def on_epoch_end(self, epoch, logs=None):
        current_beta = float(self.model.beta.numpy())
        print(f" [KL Annealing] Epoch {epoch + 1}: beta = {current_beta:.6f} / {self.beta_target}")
```

Tabel 4.6 adalah potongan program untuk kelas *KLANnealingCallback*. Kelas *KLANnealingCallback* mewarisi *tf.keras.callbacks.Callback* dan mengimplementasikan strategi *KL annealing* melalui tiga *method*:

- **__init__** menerima dua parameter yaitu *beta_target* yang merupakan nilai beta akhir hasil hyperparameter tuning, dan *annealing_epochs* yang menentukan durasi fase annealing.
- **on_epoch_begin** dipanggil otomatis oleh Keras di awal setiap *epoch*, di sinilah logika *annealing* berjalan. Nilai progress dihitung sebagai rasio *epoch* saat ini terhadap *annealing_epochs*, dibatasi maksimum 1.0 agar beta tidak melampaui *beta_target* setelah fase *annealing* selesai. Nilai beta baru kemudian ditetapkan ke model melalui *self.model.beta.assign(beta_now)* operasi ini memanfaatkan sifat *mutable* dari *tf.Variable* sehingga modifikasi

terjadi *in-place* di dalam *computational graph* tanpa membuat tensor baru maupun memicu *retracing*.

- **on_epoch_end** hanya bertugas mencetak nilai beta aktif sebagai informasi monitoring selama proses pelatihan berlangsung.

Tabel 4.7. Kelas RSVD

```
class RSVD:
    def __init__(self, n_factors=50, learning_rate=0.001,
lambda_reg=0.001, epochs=100, patience=5):
        self.k = n_factors          # Jumlah dimensi laten
        self.eta = learning_rate    # Kecepatan belajar
        self.lam = lambda_reg       # Penalti regularisasi untuk
mencegah overfitting
        self.epochs = epochs
        self.patience = patience   # Jumlah epoch tanpa
perbaikan val MSE sebelum dihentikan
        self.loss_history = []      # Menyimpan nilai Total
Loss (MSE + L2 Penalty) setiap epoch

    def fit(self, Z, val_ratio=0.1, random_seed=42):
        m, n = Z.shape             # m = Jumlah pengguna, n = Jumlah film
        np.random.seed(random_seed)

        # Ambil koordinat semua rating yang ada nilainya
        rated_indices = np.argwhere(Z > 0)
        n_val = int(len(rated_indices) * val_ratio)

        # Pilih indeks validation secara acak tanpa penggantian
        val_chosen = np.random.choice(len(rated_indices),
n_val, replace=False)
        val_idx = rated_indices[val_chosen]
        val_rows = val_idx[:, 0]
        val_cols = val_idx[:, 1]

        # Simpan nilai rating asli untuk evaluasi validation
        val_true = Z[val_rows, val_cols].copy()
```

```

        # Buat training matrix: salin Z lalu hapus rating
validation
        Z_train = Z.copy()
        Z_train[val_rows, val_cols] = 0.0

        print(f"[INFO] Total rating      : {len(rated_indices)}")
        print(f"[INFO] Training ratings : {len(rated_indices) -
n_val}")
        print(f"[INFO] Validation ratings: {n_val:,}")

        # Hitung rata-rata global HANYA dari rating training
(val sudah dihapus)
        nonzero_ratings = Z_train[Z_train > 0]
        self.mu = np.mean(nonzero_ratings) if
len(nonzero_ratings) > 0 else 0

        # Bias user (pemberi rating) dan item (film)
        self.b_u = np.zeros(m)
        self.b_i = np.zeros(n)

        # Inisialisasi U dan V dengan nilai kecil
        self.U = np.random.normal(scale=0.1, size=(m, self.k))
        self.V = np.random.normal(scale=0.1, size=(n, self.k))

        # Inisialisasi Sigma dengan Matriks Identitas
        # Mencegah error gradient terperangkap mendekati 0 di
epoch awal
        self.Sigma = np.eye(self.k)

        # Variabel untuk early stopping – memantau validation
MSE
        best_val_mse = float('inf')
        no_improve = 0

        # Snapshot bobot terbaik (disimpan saat val MSE mencapai
nilai terendah)
        best_U, best_V, best_Sigma = None, None, None

```

```

best_b_u, best_b_i = None, None

for epoch in range(self.epochs):
    for u in range(m):
        for i in range(n):
            # KUNCI UTAMA: Hanya perbarui bobot jika
            user benar-benar memberi rating
            # Gunakan Z_train (bukan Z) agar rating
            validation tidak ikut dilatih
            if Z_train[u, i] > 0:
                # Prediksi rating: Bias Global + Bias
                User + Bias Item + (U * Sigma * V^T)
                dot_product = np.dot(np.dot(self.U[u,
:], self.Sigma), self.V[i, :].T)
                pred = self.mu + self.b_u[u] +
                self.b_i[i] + dot_product

                # Hitung jarak error (Asli dikurangi
                Prediksi)
                e_ui = Z_train[u, i] - pred

                # Update Bias menggunakan Gradient
                Descent & L2 Regularization
                self.b_u[u] += self.eta * (e_ui -
                self.lam * self.b_u[u])
                self.b_i[i] += self.eta * (e_ui -
                self.lam * self.b_i[i])

                # Update Matriks Laten
                for k in range(self.k):
                    U_uk = self.U[u, k]
                    V_ik = self.V[i, k]
                    Sigma_kk = self.Sigma[k, k]

                    # Memperbarui bobot U, V, dan Sigma
                    berdasarkan gradien error

```



```

        self.U[u, k] += self.eta * (e_ui *
Sigma_kk * V_ik - self.lam * U_uk)
        self.V[i, k] += self.eta * (e_ui *
Sigma_kk * U_uk - self.lam * V_ik)
        self.Sigma[k, k] += self.eta * (e_ui
* U_uk * V_ik)

    # Rekonstruksi matriks penuh
    bias_matrix = self.mu + self.b_u[:, np.newaxis] +
self.b_i[np.newaxis, :]
    latent_matrix = np.dot(np.dot(self.U, self.Sigma),
self.V.T)
    reconstructed_Z = bias_matrix + latent_matrix

    # Hitung training MSE (hanya pada rating training,
sel kosong dan val diabaikan)
    train_mask = (Z_train > 0)
    current_mse = np.sum(np.square(Z_train[train_mask] -
reconstructed_Z[train_mask])) / np.sum(train_mask)

    # Hitung validation MSE (pada rating yang disisihkan
di awal)
    val_pred = reconstructed_Z[val_rows, val_cols]
    val_mse = np.mean(np.square(val_true - val_pred))

    #  $\lambda * (||U||^2_F + ||V||^2_F)$ 
    # Sigma tidak diregularisasi (lihat Catatan Desain
di docstring)
    l2_penalty = self.lam * (np.sum(np.square(self.U)) +
np.sum(np.square(self.V)))

    # Total Loss = Training MSE + L2 Penalty (rumus 8
proposal)
    current_loss = current_mse + l2_penalty
    self.loss_history.append(current_loss)

    # Print progress setiap epoch

```

```

        print(f"Epoch {epoch+1:03d}/{self.epochs} | Train
MSE: {current_mse:.6f} | Val MSE: {val_mse:.6f} | L2:
{l2_penalty:.6f} | Total Loss: {current_loss:.6f}")
        # Validation MSE dipakai karena training MSE selalu
turun
        # dan tidak mencerminkan performa di data yang belum
dilihat
        if val_mse < best_val_mse:
            best_val_mse = val_mse
            no_improve = 0

            # Simpan snapshot bobot terbaik saat ini
            best_U = self.U.copy()
            best_V = self.V.copy()
            best_Sigma = self.Sigma.copy()
            best_b_u = self.b_u.copy()
            best_b_i = self.b_i.copy()
        else:
            no_improve += 1
            if no_improve >= self.patience:
                print(f"\n[Early Stopping] Tidak ada
perbaikan Val MSE selama {self.patience} epoch.")
                print(f"[Early Stopping] Berhenti di epoch
{epoch+1}. Best Val MSE: {best_val_mse:.6f}")

                # Kembalikan bobot ke snapshot terbaik
                self.U = best_U
                self.V = best_V
                self.Sigma = best_Sigma
                self.b_u = best_b_u
                self.b_i = best_b_i
                break

```

Tabel 4.7 adalah potongan program untuk kelas RSVD. Kelas RSVD mengimplementasikan algoritma faktorisasi matriks dengan dua *method*:

- `__init__` menginisialisasi lima hyperparameter model:
 - `n_factors` (jumlah dimensi laten k)

- `learning_rate` (kecepatan belajar η)
- `lambda_reg` (koefisien regularisasi L2 λ)
- `epochs` (batas maksimum iterasi)
- `patience` (toleransi *early stopping*).

Atribut `loss_history` diinisialisasi sebagai *list* kosong untuk merekam *total loss* setiap *epoch*.

- **fit** menjalankan seluruh pipeline pelatihan RSVD yang terdiri dari empat tahap:
 - **validation split**, sebanyak `val_ratio` dari total rating disisihkan secara acak sebagai data validasi, dan matriks training Z_{train} dibuat dengan menghapus posisi rating validasi tersebut. Pemisahan ini diperlukan agar *early stopping* dapat memantau performa pada data yang tidak dilihat selama pelatihan.
 - **inisialisasi parameter**, bias global μ dihitung dari rata-rata rating training saja, bias *user* b_u dan bias *item* b_i diinisialisasi nol, matriks faktor laten U dan V diinisialisasi dengan nilai acak kecil, sedangkan Σ diinisialisasi sebagai matriks identitas untuk mencegah *vanishing gradient* di *epoch-epoch* awal akibat nilai Σ yang terlalu kecil.
 - **training loop**, untuk setiap *epoch*, model melakukan iterasi pada seluruh pasangan (*user*, *item*) dan hanya memperbarui parameter pada posisi yang memiliki rating eksplisit. Prediksi dihitung menggunakan **persamaan (9)**, *error* dihitung dengan **persamaan (10) dan (11)**, lalu bias dan matriks laten diperbarui menggunakan SGD dengan regularisasi L2 menggunakan **persamaan (12) – (15)**. Perlu diperhatikan bahwa Σ tidak dikenai regularisasi L2 karena eksperimen awal menunjukkan bahwa meregularisasi Σ menyebabkan *weight collapse* di mana nilai Σ mendekati nol secara kaskade dan menghambat gradien U dan V .
 - **early stopping**, di akhir setiap *epoch*, *validation MSE* dihitung dan dibandingkan dengan nilai terbaik sebelumnya. Jika tidak ada perbaikan selama *patience epoch* berturut-turut, pelatihan dihentikan dan bobot dikembalikan ke *snapshot* terbaik yang telah disimpan.

4.1.2 Program Preprocessing Data

Notebook DataPreprocessing.ipynb bertanggung jawab atas seluruh tahap persiapan data sebelum proses pelatihan model dimulai. *Notebook* ini diorganisasi ke dalam lima seksi utama dan menghasilkan empat *file* keluaran berformat .npy

yang digunakan oleh *notebook-notebook* berikutnya dalam alur kerja sistem. Potongan kode *preprocessing data* terdapat pada **tabel 4.8 - 4.15**

1. File Paths

Seksi ini mendefinisikan dua variabel *file path* yang digunakan secara konsisten di seluruh *notebook*. *raw_data_path* menunjuk ke *file u.data* dari dataset *MovieLens 100K*, dan *output_dir* menunjuk ke folder *TrainTest* sebagai tujuan penyimpanan hasil *preprocessing*. Pendefinisian jalur secara terpusat di awal *notebook* memudahkan adaptasi apabila struktur direktori berubah. Kode seksi ini ditampilkan pada **Tabel 4.8**.

Tabel 4.8. Cell *File Paths*

```
# Path ke file data mentah MovieLens 100K
raw_data_path = '../Data/ml-100k/u.data'

# Path folder output untuk menyimpan hasil preprocessing
output_dir = '../Data/TrainTest'
```

2. Load Data

File u.data dibaca menggunakan *pd.read_csv* dengan pemisah *tab* dan empat nama kolom eksplisit yaitu *user_id*, *item_id*, *rating*, dan *timestamp*. Kolom *timestamp* dimuat tetapi tidak digunakan dalam pemodelan karena sistem rekomendasi yang dibangun tidak memodelkan preferensi berbasis waktu. Kode seksi ini ditampilkan pada **Tabel 4.9**.

Tabel 4.9. Cell *Load Data*

```
# Kolom pada file u.data: user_id, item_id, rating,
timestamp
# File menggunakan tab sebagai separator
column_names = ['user_id', 'item_id', 'rating',
'timestamp']
df = pd.read_csv(raw_data_path, sep='\t',
names=column_names)
```

3. Matriks Interaksi *User-Item*

Data tabular diubah menjadi matriks *user-item* dua dimensi menggunakan *pivot*, dengan *user_id* sebagai indeks baris, *item_id* sebagai indeks kolom, dan nilai *rating* sebagai isi sel. Pasangan *user-item* yang tidak memiliki *rating* diisi dengan nilai nol melalui *fillna(0)*, sehingga sel kosong dapat dibedakan dari *rating* terendah setelah normalisasi. Kode seksi ini ditampilkan pada **Tabel 4.10**.

Tabel 4.10. Cell Matriks Interaksi *User-Item*

```
# Pivot tabel menjadi matriks user x item
# Sel yang tidak memiliki rating diisi dengan 0
interaction_matrix = df.pivot(
    index='user_id',
    columns='item_id',
    values='rating'
).fillna(0)
```

4. Normalisasi Rating

Seluruh nilai rating dinormalisasi ke rentang [0, 1] dengan membaginya terhadap nilai rating maksimum ($MAX_RATING = 5.0$). Strategi normalisasi ini dipilih agar sel bernilai nol yang merepresentasikan ketiadaan rating tetap bernilai nol setelah normalisasi, sehingga dapat dibedakan dari rating terendah yang bernilai 0,2 (yaitu $1/5$). Metode `.values` digunakan untuk mengekstrak representasi *NumPy array* dari *DataFrame* sebelum normalisasi dilakukan. Kode seksi ini ditampilkan pada Tabel 4.11.

Tabel 4.11. Cell Normalisasi Rating

```
# Normalisasi rating ke rentang [0, 1] dengan membagi
nilai maksimum rating (5.0)
MAX_RATING = 5.0

normalized_matrix = interaction_matrix.values / MAX_RATING
```

5. Train / Validation / Test Split

Split dilakukan pada level rating individual sehingga setiap pengguna dapat muncul di ketiga partisi. Tabel 4.12 menampilkan konfigurasi rasio *split* dan pengambilan koordinat rating, di mana seluruh koordinat rating yang bernilai non-nol diidentifikasi menggunakan `np.nonzero` lalu jumlah rating untuk masing-masing partisi dihitung berdasarkan proporsi 70/10/20. Tabel 4.13 menampilkan proses pengacakan indeks menggunakan `np.random.permutation` dengan *seed* tetap ($RANDOM_SEED = 42$) untuk menjamin reproduksibilitas, kemudian indeks dipotong menjadi tiga bagian sesuai proporsi dan koordinat masing-masing partisi diekstrak. Tabel 4.14 menampilkan pembangunan matriks terpisah untuk setiap partisi. Matriks *train* dimulai dari salinan matriks penuh kemudian posisi *validation* dan *test* dihapus menjadi nol, sedangkan matriks *validation* dan *test* masing-masing dibangun dari matriks nol yang kemudian diisi hanya pada posisi rating yang relevan.

Tabel 4.12. Cell Konfigurasi Rasio *Split*

```
# Konfigurasi split
VAL_RATIO = 0.10 # 10% untuk validasi
TEST_RATIO = 0.20 # 20% untuk test
RANDOM_SEED = 42

# Ambil koordinat semua rating yang ada nilainya
all_rows, all_cols = np.nonzero(normalized_matrix)
total_ratings = len(all_rows)

n_test = int(total_ratings * TEST_RATIO)
n_val = int(total_ratings * VAL_RATIO)
n_train = total_ratings - n_test - n_val
```

Tabel 4.13. *Cell Shuffle* dan Potong Indeks

```
# Acak seluruh indeks rating dengan seed tetap untuk
reproduksibilitas
np.random.seed(RANDOM_SEED)
shuffled_idx = np.random.permutation(total_ratings)

# Potong indeks menjadi tiga bagian berdasarkan proporsi
test_idx = shuffled_idx[:n_test]
val_idx = shuffled_idx[n_test:n_test + n_val]
train_idx = shuffled_idx[n_test + n_val:]

# Ambil koordinat (baris, kolom) masing-masing partisi
test_rows, test_cols =
all_rows[test_idx], all_cols[test_idx]
val_rows, val_cols =
all_rows[val_idx], all_cols[val_idx]
train_rows, train_cols = all_rows[train_idx],
all_cols[train_idx]
```

Tabel 4.14. Cell Bangun Matriks *Train, Validation, dan Test*

```
# Bangun matriks train: mulai dari matriks penuh, hapus
rating val dan test
```

```

train_data = normalized_matrix.copy().astype(np.float32)
train_data[val_rows, val_cols] = 0.0
train_data[test_rows, test_cols] = 0.0

# Bangun matriks validation: mulai dari nol, isi hanya
rating val
val_data = np.zeros(normalized_matrix.shape,
dtype=np.float32)
val_data[val_rows, val_cols] = normalized_matrix[val_rows,
val_cols]

# Bangun matriks test: mulai dari nol, isi hanya rating
test
test_data = np.zeros(normalized_matrix.shape,
dtype=np.float32)
test_data[test_rows, test_cols] =
normalized_matrix[test_rows, test_cols]

```

6. *Save Data*

Keempat matriks disimpan dalam format .npy menggunakan np.save ke folder output_dir. train_data.npy (70%), val_data.npy (10%), dan test_data.npy (20%) digunakan oleh *Training.ipynb* dan *Testing.ipynb*, sedangkan normalized_matrix.npy disimpan secara khusus untuk *CrossValidation.ipynb* agar setiap *fold* dapat membuat *split*-nya sendiri dari matriks rating yang utuh. Kode seksi ini ditampilkan pada **Tabel 4.15**.

Tabel 4.15. Cell *Save Data*

```

# Simpan matriks train (70%): dipakai untuk melatih VAE dan
RSVD di Training.ipynb
np.save(os.path.join(output_dir, 'train_data'),
train_data)

# Simpan matriks validation (10%): dipakai untuk mencari
alpha ensemble di Testing.ipynb
np.save(os.path.join(output_dir, 'val_data'), val_data)

# Simpan matriks test (20%): hanya disentuh satu kali untuk
pelaporan performa akhir
np.save(os.path.join(output_dir, 'test_data'), test_data)

```

```
# Simpan matriks rating penuh sebelum di-split
# Dipakai oleh CrossValidation.ipynb agar setiap fold bisa
membuat split-nya sendiri
np.save(os.path.join(output_dir, 'normalized_matrix'),
normalized_matrix)
```

4.1.3 Program *Hyperparameter Tuning*

HyperParameterTuningOptuna.ipynb merupakan *notebook* yang mengelola proses pencarian hyperparameter optimal untuk model VAE dan RSVD. Secara keseluruhan, *notebook* ini terdiri dari empat seksi utama yang dijalankan secara berurutan: impor *library*, konfigurasi *path*, pemuatan data dan model, serta proses *tuning* untuk masing-masing model.

1. Import Library

Tabel 4.16. Cell *Import Library*

```
import numpy as np
import tensorflow as tf
import itertools
import sys
import os
import gc
import json
import optuna
```

Cell pertama pada seksi ini, ditampilkan pada **Tabel 4.16**, mengimpor seluruh *library* yang dibutuhkan selama proses *hyperparameter tuning*. *Library* numpy dan tensorflow digunakan untuk operasi numerik dan pelatihan model VAE. *Library* itertools digunakan untuk menghasilkan seluruh kombinasi hyperparameter pada proses *grid search* RSVD. *Library* sys dan os digunakan untuk manipulasi *path* sistem, gc untuk manajemen memori secara eksplisit, json untuk membaca dan menyimpan hasil *tuning*, serta optuna sebagai *framework* optimasi hyperparameter berbasis TPE untuk VAE.

2. File Path

Tabel 4.17. Cell *Path Folder*


```

scripts_folder = os.path.abspath(os.path.join '..',
'Scripts'))
data_folder = os.path.abspath(os.path.join '..', 'Data',
'TrainTest'))
hyper_parameter_folder =
os.path.abspath(os.path.join '..', 'Data',
'HyperParameters'))
sys.path.append(scripts_folder)

```

Cell definisi *path* folder ditampilkan pada **Tabel 4.17**. Cell mendefinisikan tiga variabel *path* yaitu *scripts_folder* yang menunjuk ke direktori Scripts tempat model.py berada, *data_folder* yang menunjuk ke direktori TrainTest tempat data hasil *preprocessing* tersimpan, dan *hyper_parameter_folder* yang menunjuk ke direktori HyperParameters sebagai lokasi penyimpanan hasil *tuning*. Pada akhir cell *scripts_folder* ditambahkan ke *sys.path* agar modul model.py dapat diimpor secara langsung pada cell berikutnya.

3. Import Data & Model

Tabel 4.18. Cell *Import Data & Model*

```

from model import Encoder, Decoder, VAE, RSVD

train_data_path = os.path.join(data_folder,
'train_data.npy')

# Import training data
try:
    train_data =
np.load(train_data_path).astype(np.float32)
    train_data_tf = tf.constant(train_data,
dtype=tf.float32) # Mencegah tensorflow duplikasi data
tiap iterasi (memory leak)
    # Mengambil jumlah item
    num_items = train_data.shape[1]
    print(f"Data latih berhasil dimuat.")
    print(f"    Dimensi data: {train_data.shape} (Pengguna
x Film)")
except FileNotFoundError:
    print("Error: File 'train_data.npy' tidak ditemukan.
Pastikan Anda sudah menjalankan preprocessing.")

```

Seksi ini memuat kelas model dan data latih melalui dua cell yang ditampilkan bersama pada **Tabel 4.18**. Cell pertama mengimpor kelas Encoder, Decoder, VAE, dan RSVD dari model.py. Cell kedua memuat data latih dari *file* train_data.npy ke dalam *array* NumPy bertipe float32. Data yang sama kemudian dikonversi menjadi tf.constant dan disimpan dalam variabel train_data_tf untuk mencegah TensorFlow menduplikasi data pada setiap iterasi pelatihan yang berpotensi menyebabkan *memory leak*. Selain itu, jumlah item num_items diekstrak dari dimensi kolom data latih dan akan digunakan sebagai ukuran lapisan keluaran *decoder* VAE.

4. Hyperparameter Tuning

Seksi ini merupakan inti dari notebook dan terbagi menjadi dua sub-seksi yaitu *tuning* VAE menggunakan Optuna dan *tuning* RSVD menggunakan *grid search exhaustif*.

a. Tuning VAE (Optuna TPE)

Sub-seksi VAE terdiri dari lima cell yang dijalankan secara berurutan. Cell pertama, ditampilkan pada **Tabel 4.19**, mendefinisikan search space hyperparameter VAE dalam bentuk dictionary vae_search_space. Ruang pencarian mencakup enam dimensi:

- i. latent_dim dengan empat nilai kandidat (20, 50, 100, 200)
- ii. learning_rate dengan lima nilai kandidat (1×10^{-4} hingga 1×10^{-2})
- iii. batch_size dengan tiga nilai kandidat (64, 128, 256)
- iv. dropout_rate dengan empat nilai kandidat (0.1 hingga 0.4)
- v. hidden_dims dengan empat konfigurasi arsitektur lapisan tersembunyi
- vi. beta dengan tujuh nilai kandidat (0.01 hingga 1.0).

Jumlah trial Optuna ditetapkan sebesar 200 melalui konstanta N_TRIALS_VAE.

Tabel 4.19. Cell Definisi Ruang Pencarian VAE

```
# Ruang pencarian hyperparameter VAE untuk Optuna
vae_search_space = {
    'latent_dim': [20, 50, 100, 200],
    'learning_rate': [1e-4, 5e-4, 1e-3, 5e-3, 1e-2],
    'batch_size': [64, 128, 256],
    'dropout_rate': [0.1, 0.2, 0.3, 0.4],
    'hidden_dims': [
        [512, 256],
        [512, 256, 128],
        [1024, 512],
```

```

        [1024, 512, 256],
    ],
    'beta': [0.01, 0.05, 0.1, 0.3, 0.5, 0.8, 1.0],
}

# Jumlah trial yang akan dijalankan Optuna
N_TRIALS_VAE = 200

```

Cell kedua, ditampilkan pada **Tabel 4.20**, mendefinisikan fungsi objektif `vae_objective` yang dipanggil Optuna pada setiap *trial*. Fungsi ini menerima objek *trial* dari Optuna dan menggunakan metode `suggest_categorical` untuk memilih nilai dari masing-masing dimensi *search space*. Karena `suggest_categorical` hanya menerima tipe primitif, nilai `hidden_dims` yang berupa *list* dikonversi terlebih dahulu ke *string* sebelum dipilih, lalu dikembalikan ke *list* menggunakan `eval`. Setelah hyperparameter dipilih, sesi TensorFlow dibersihkan, model VAE dibangun dan dikompilasi, kemudian dilatih selama 30 *epoch*. Nilai `reconstruction_loss` pada *epoch* terakhir dikembalikan sebagai nilai objektif yang akan diminimalkan oleh Optuna. Penggunaan `reconstruction_loss` sebagai kriteria seleksi dilakukan untuk menghindari bias terhadap nilai *beta* yang kecil, karena *total loss* akan lebih kecil secara artifisial ketika *beta* bernilai rendah meskipun kualitas rekonstruksi tidak lebih baik.

Tabel 4.20. Cell Fungsi Objektif VAE

```

def vae_objective(trial):
    latent_dim =
trial.suggest_categorical('latent_dim',
vae_search_space['latent_dim'])
    learning_rate =
trial.suggest_categorical('learning_rate',
vae_search_space['learning_rate'])
    batch_size =
trial.suggest_categorical('batch_size',
vae_search_space['batch_size'])
    dropout_rate =
trial.suggest_categorical('dropout_rate',
vae_search_space['dropout_rate'])

```

```

        beta = trial.suggest_categorical('beta',
vae_search_space['beta'])

        hidden_dims_str =
trial.suggest_categorical('hidden_dims', [str(h) for
h in vae_search_space['hidden_dims']])
        hidden_dims = eval(hidden_dims_str)

        tf.keras.backend.clear_session()
        tf.compat.v1.reset_default_graph()

        encoder = Encoder(hidden_dims=hidden_dims,
latent_dim=latent_dim, dropout_rate=dropout_rate)
        decoder = Decoder(hidden_dims=hidden_dims[:-1],
output_dim=num_items)
        vae = VAE(encoder, decoder, beta=beta)

        dummy_output = vae(train_data_tf[:1])

        optimizer =
tf.keras.optimizers.Adam(learning_rate=learning_rate
)

        vae.compile(optimizer=optimizer,
run_eagerly=True)

        history = vae.fit(
            train_data_tf, train_data_tf,
            epochs = 30,
            batch_size = batch_size,
            verbose = 0
        )

        reconstruction_loss =
history.history['reconstruction_loss'][-1]

        del encoder, decoder, vae, history, optimizer,
dummy_output

```

```
gc.collect()

tf.keras.backend.clear_session()

return reconstruction_loss
```

Cell ketiga, ditampilkan pada **Tabel 4.21**, membuat atau melanjutkan *Optuna study* menggunakan penyimpanan SQLite (*optuna_vae_study.db*). Penggunaan *load_if_exists=True* memungkinkan *study* dilanjutkan secara otomatis apabila proses *notebook* terputus sebelum seluruh trial selesai. Cell ini juga menghitung jumlah trial yang sudah selesai (*n_completed*) dan trial yang tersisa (*n_remaining*) sebelum proses *tuning* dilanjutkan.

Tabel 4.21. Cell Inisialisasi *Study* Optuna

```
study_vae = optuna.create_study(
    study_name = 'vae_tuning',
    direction = 'minimize',
    storage = f'sqlite:///vae_study_db',
    load_if_exists = True    # otomatis resume jika
study sudah ada
)

n_completed = len([t for t in study_vae.trials if
t.state == optuna.trial.TrialState.COMPLETE])
n_remaining = max(0, N_TRIALS_VAE - n_completed)
```

Cell keempat, ditampilkan pada **Tabel 4.22**, menjalankan proses optimasi dengan memanggil *study_vae.optimize*. Sebuah *callback* *print_trial_callback* didefinisikan untuk mencetak hasil setiap *trial* secara langsung, termasuk penanda [NEW BEST] apabila *trial* tersebut menghasilkan *reconstruction loss* terendah baru. Optimasi hanya dijalankan apabila masih terdapat trial yang tersisa (*n_remaining > 0*), sehingga cell ini aman untuk dieksekusi ulang tanpa menimpa *trial* yang sudah selesai.

Tabel 4.22. Cell Eksekusi *Tuning* VAE

```
def print_trial_callback(study, trial):
    print(f" Trial {trial.number+1:>3} | Recon
Loss: {trial.value:.6f} | Params: {trial.params}")
    if study.best_trial.number == trial.number:
```

```

        print(f"[NEW BEST] Reconstruction Loss turun
ke {trial.value:.6f}")

if n_remaining > 0:
    print(f"[INFO] Memulai {n_remaining} trial
Optuna (TPE)...\n")
    study_vae.optimize(
        vae_objective,
        n_trials = n_remaining,
        callbacks = [print_trial_callback],
        gc_after_trial = True    # bersihkan memori
setelah setiap trial
    )
else:
    print("[INFO] Semua trial sudah selesai. Tidak
ada yang perlu dijalankan.")

```

Cell kelima, ditampilkan pada **Tabel 4.23**, menyimpan hasil *tuning* terbaik VAE ke *file* JSON `tuning_progress_vae.json`. Sebelum disimpan, nilai `hidden_dims` dikonversi kembali dari *string* ke *list* menggunakan `eval`. Format JSON yang dihasilkan seragam dengan format yang dibaca oleh `Training.ipynb`, `CrossValidation.ipynb`, dan `Testing.ipynb`, sehingga tidak diperlukan modifikasi pada *notebook-notebook* tersebut.

Tabel 4.23. Cell Simpan Hasil Terbaik *Tuning* VAE

```

best_params_raw = study_vae.best_params.copy()
best_params_raw['hidden_dims'] =
eval(best_params_raw['hidden_dims'])

with open(vae_progress_file, 'w') as f:
    json.dump({
        'best_loss': study_vae.best_value,
        'best_params': best_params_raw,
    }, f, indent=4)

```

b. *Tuning* RSVD (Grid Search)

Sub-seksi RSVD terdiri dari tiga cell. Cell pertama, ditampilkan pada **Tabel 4.24**, mendefinisikan *grid search* dalam bentuk *dictionary* `rsvd_param_grid` yang mencakup tiga dimensi:

- i. `n_factors` dengan tiga kandidat (10, 20, 50)
- ii. `learning_rate` dengan empat kandidat (0.001 hingga 0.02)
- iii. `lambda_reg` dengan empat kandidat (0.001 hingga 0.02).

Seluruh kombinasi dihasilkan menggunakan `itertools.product`, menghasilkan 48 kombinasi yang akan dievaluasi secara exhaustif.

Tabel 4.24. Cell Definisi Ruang Pencarian RSVD

```
rsvd_param_grid = {
    'n_factors': [10, 20, 50],
    'learning_rate': [0.001, 0.005, 0.01, 0.02],
    'lambda_reg': [0.001, 0.005, 0.01, 0.02]
}

rsvd_keys, rsvd_values =
zip(*rsvd_param_grid.items())
rsvd_combinations = [dict(zip(rsvd_keys, v)) for v
in itertools.product(*rsvd_values)]
```

Cell kedua terbagi menjadi dua sub-cell yaitu pemuatan *progress* dan *looping* utama. Sub-cell pemuatan *progress*, ditampilkan pada **Tabel 4.25**, memuat *file* `tuning_progress_rsvd.json` apabila sudah tersedia. Dari *file* tersebut, nilai `best_error`, `best_params`, dan daftar `completed_indices` dipulihkan agar proses dapat dilanjutkan dari titik terakhir tanpa mengulang kombinasi yang sudah dievaluasi.

Tabel 4.25. Cell Load Progress *Tuning* RSVD

```
rsvd_progress_file =
os.path.join(hyper_parameter_folder,
'tuning_progress_rsvd.json')

if os.path.exists(rsvd_progress_file):
    with open(rsvd_progress_file, 'r') as f:
        progress_data = json.load(f)
```

```

        best_rsvd_error = progress_data['best_error']
        best_rsvd_params = progress_data['best_params']
        completed_indices =
progress_data['completed_indices']

        print(f"\n[INFO] Melanjutkan sesi RSVD
sebelumnya...")
        print(f"[INFO] {len(completed_indices)}
kombinasi sudah dievaluasi.")
        print(f"[INFO] MSE terbaik saat ini:
{best_rsvd_error:.4f}")
    else:
        best_rsvd_error = float('inf')
        best_rsvd_params = None
        completed_indices = []

```

Sub-cell *looping*, ditampilkan pada **Tabel 4.26**, mengiterasi seluruh 48 kombinasi. Untuk setiap kombinasi yang belum dikerjakan, model RSVD diinisialisasi dan dilatih selama 15 *epoch*, kemudian nilai MSE pada *epoch* terakhir (`loss_history[-1]`) digunakan sebagai kriteria evaluasi. Apabila MSE yang diperoleh lebih rendah dari nilai terbaik sebelumnya, model dinyatakan sebagai model terbaik baru dan seluruh komponennya disimpan sebagai *file* .*np*y. Setiap selesai mengevaluasi satu kombinasi, *progress* langsung disimpan ke *file* JSON sehingga proses aman untuk diinterupsi kapan saja.

Tabel 4.26. Cell Eksekusi *Tuning* RSVD

```

for i, params in enumerate(rsvd_combinations):
    if i in completed_indices:
        continue

    rsvd = RSVD(
        n_factors=params['n_factors'],
        learning_rate=params['learning_rate'],
        lambda_reg=params['lambda_reg'],
        epochs=15 # Epoch kecil untuk tuning

```



```

)

rsvd.fit(train_data)

total_loss_error = rsvd.loss_history[-1]

if total_loss_error < best_rsvd_error:
    best_rsvd_error = total_loss_error
    best_rsvd_params = params
    best_rsvd_model = rsvd

    np.save(os.path.join(hyper_parameter_folder,
'best_mu.npy'), np.array(rsvd.mu))
    np.save(os.path.join(hyper_parameter_folder,
'best_b_u.npy'), rsvd.b_u)
    np.save(os.path.join(hyper_parameter_folder,
'best_b_i.npy'), rsvd.b_i)
    np.save(os.path.join(hyper_parameter_folder,
'best_U.npy'), rsvd.U)
    np.save(os.path.join(hyper_parameter_folder,
'best_Sigma.npy'), rsvd.Sigma)
    np.save(os.path.join(hyper_parameter_folder,
'best_V.npy'), rsvd.V)

    completed_indices.append(i)
    with open(rsvd_progress_file, 'w') as f:
        json.dump({
            'best_error': best_rsvd_error,
            'best_params': best_rsvd_params,
            'completed_indices': completed_indices
        }, f)

del rsvd
gc.collect()

```

4.1.4 Program Training

Notebook Training.ipynb mengimplementasikan proses pelatihan akhir untuk model VAE dan RSVD menggunakan hyperparameter terbaik yang telah diperoleh dari tahap *hyperparameter tuning*. *Notebook* ini terdiri dari lima seksi utama, yaitu impor *library*, konfigurasi jalur direktori, load data dan kelas model, proses *training* VAE dan RSVD beserta penyimpanan artefaknya, serta visualisasi kurva konvergensi. Implementasi pada *notebook* ini ditampilkan pada **Tabel 4.27** hingga **Tabel 4.35**.

1. Import Library

Tabel 4.27 menampilkan cell impor *library* yang digunakan pada *notebook* ini. Cell ini mengimpor seluruh *library* yang dibutuhkan selama proses pelatihan. *Library* *os* dan *sys* digunakan untuk manajemen jalur sistem, *json* untuk membaca file hyperparameter hasil tuning, *numpy* untuk operasi numerik, *tensorflow* sebagai *framework* pelatihan VAE, *matplotlib.pyplot* untuk visualisasi kurva konvergensi, serta *datetime* untuk pemberian *timestamp* pada file keluaran.

Tabel 4.27. Cell *Import Library* Pada Training.ipynb

```
import os
import sys
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime
```

2. Konfigurasi Direktori

Tabel 4.28 menampilkan cell konfigurasi jalur direktori yang digunakan sepanjang *notebook*. Cell ini mendefinisikan seluruh jalur direktori yang digunakan selama proses pelatihan. Variabel *scripts_path* mengarah ke direktori *Scripts* tempat *model.py* disimpan, dan ditambahkan ke *sys.path* agar kelas-kelas model dapat diimpor. Variabel *data_dir* mengarah ke direktori data latih, *hyperparameter_dir* ke direktori penyimpanan hasil tuning, *save_dir* ke direktori penyimpanan bobot model terlatih, *loss_history_dir* ke direktori riwayat loss, dan *visualization_dir* ke direktori penyimpanan gambar visualisasi.

Tabel 4.28. Cell Konfigurasi Direktori Pada Training.ipynb

```

scripts_path = os.path.abspath(os.path.join('..',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir = os.path.abspath(os.path.join('..', 'Data',
'TrainTest'))
hyperparameter_dir = os.path.abspath(os.path.join('..',
'Data', 'HyperParameters'))
save_dir = os.path.abspath(os.path.join('..', 'Data',
'SavedModels'))
loss_history_dir = os.path.abspath(os.path.join('..',
'Data', 'LossHistory'))
visualization_dir = os.path.abspath(os.path.join('..',
'Data', 'Visualization'))

```

3. Load Data & Model

Tabel 4.29 menampilkan cell pemuatan data latih dan impor kelas model yang digabungkan menjadi satu blok persiapan. Cell ini memuat matriks rating data latih dari file `train_data.npy` ke dalam variabel `train_data` bertipe `float32`. Dimensi kolom matriks tersebut diekstrak sebagai variabel `num_items` yang merepresentasikan jumlah total item (film) dan akan digunakan sebagai dimensi output layer VAE. Selanjutnya, kelas-kelas Encoder, Decoder, VAE, RSVD, dan `KLAnnealingCallback` diimpor dari `model.py`.

Tabel 4.29. Cell Load Data Latih dan Import Kelas Pada Training.ipynb

```

# Load matriks rating (data latih)
train_data = np.load(os.path.join(data_dir,
'train_data.npy')).astype(np.float32)

# Mengambil jumlah item (film) untuk layer output VAE
num_items = train_data.shape[1]

from model import Encoder, Decoder, VAE, RSVD,
KLAnnealingCallback

```

4. Training VAE

Seksi pelatihan VAE terdiri dari tiga tahap yang ditampilkan pada **Tabel 4.30, Tabel 4.31, dan Tabel 4.32.**

Tabel 4.30. Cell Import *Hyperparameter* dan Konstruksi VAE Pada Training.ipynb

```
vae_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f)['best_params']

encoder =
Encoder(hidden_dims=best_vae_params['hidden_dims'],
latent_dim=best_vae_params['latent_dim'],
dropout_rate=best_vae_params['dropout_rate'])
decoder =
Decoder(hidden_dims=best_vae_params['hidden_dims'][:-1],
output_dim=num_items)

# Inisialisasi VAE dengan beta=0.0 agar KL annealing mulai
dari nol
final_vae = VAE(encoder, decoder, beta=0.0)

_ = final_vae(train_data[:1])

optimizer =
tf.keras.optimizers.Adam(learning_rate=best_vae_params['le
arning_rate'])
final_vae.compile(optimizer=optimizer)

# Jumlah epoch untuk menaikkan beta dari 0 ke beta_target
secara linear.
KL_ANNEALING_EPOCHS = 30

# Callback KL annealing: naikan beta bertahap dari 0 →
best_vae_params['beta']
kl_annealing = KLANnealingCallback(
    beta_target=best_vae_params['beta'],
```

```

        annealing_epochs=KL_ANNEALING_EPOCHS
    )

    # Early stopping tetap dipakai seperti sebelumnya
    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor='loss', patience=10,
        restore_best_weights=True, verbose=1
    )

```

Program pada **Tabel 4.30** ini memuat hyperparameter VAE terbaik dari file `tuning_progress_vae.json`, kemudian menggunakannya untuk membangun arsitektur model. Objek Encoder dikonstruksi dengan `hidden_dims`, `latent_dim`, dan `dropout_rate` dari hasil *tuning*, sedangkan objek Decoder dikonstruksi dengan urutan `hidden_dims` yang dibalik (`[::-1]`) untuk membentuk arsitektur simetris. Model VAE diinisialisasi dengan $\beta = 0.0$ agar nilai *beta* dimulai dari nol dan dinaikkan secara bertahap oleh `KLAnnealingCallback`. *Forward pass* awal dilakukan pada satu sampel data (`train_data[:1]`) untuk membangun bobot model sebelum kompilasi. *Optimizer* Adam dikompilasi dengan `learning_rate` dari hasil *tuning*. Nilai `KL_ANNEALING_EPOCHS` ditetapkan sebesar 30 sebagai jumlah *epoch* annealing, dan *callback* `EarlyStopping` dikonfigurasi dengan `patience=10` untuk menghentikan pelatihan apabila tidak ada perbaikan *loss* dalam 10 *epoch* berturut-turut.

Tabel 4.31. Cell *Training* VAE Pada `Training.ipynb`

```

history = final_vae.fit(
    train_data, train_data,
    epochs=500,
    batch_size=best_vae_params['batch_size'],
    callbacks=[kl_annealing, early_stopping],
    verbose=1
)

```

Program pada **Tabel 4.31** menjalankan proses pelatihan VAE menggunakan metode `fit`. Data latih digunakan sebagai masukan sekaligus target (`train_data, train_data`) sesuai dengan paradigma *autoencoder*. Jumlah *epoch* maksimum ditetapkan sebesar 500 dengan *callback* `kl_annealing` ditempatkan sebelum `early_stopping` agar nilai *beta* diperbarui terlebih dahulu sebelum *loss* dihitung pada setiap awal *epoch*.

Tabel 4.32. Cell Menyimpan Bobot dan *Loss History* VAE Pada Training.ipynb

```
final_vae.save_weights(os.path.join(save_dir,
'trained_best_vae_weights.weights.h5'))

vae_history_dict = {'loss': history.history['loss']}

with open(os.path.join(loss_history_dir,
'final_vae_loss_history.json'), 'w') as f:
    json.dump(vae_history_dict, f)
```

Program pada Tabel 4.32 menyimpan hasil pelatihan VAE. Bobot model disimpan dalam format .h5 ke direktori SavedModels dengan nama trained_best_vae_weights.weights.h5. Riwayat nilai *loss* per *epoch* disimpan dalam format JSON ke direktori LossHistory dengan nama final_vae_loss_history.json, yang akan digunakan kembali pada tahap visualisasi.

5. Training RSVD

Seksi pelatihan RSVD terdiri dari tiga tahap yang ditampilkan pada Tabel 4.33, Tabel 4.34, dan Tabel 4.35.

Tabel 4.33. Cell Load *Hyperparameter* dan Konstruksi Model RSVD Pada Training.ipynb

```
rsvd_progress_file = os.path.join(hyperparameter_dir,
'tuning_progress_rsvd.json')
with open(rsvd_progress_file, 'r') as f:
    best_rsvd_params = json.load(f)['best_params']

final_rsvd = RSVD(
    n_factors=best_rsvd_params['n_factors'],
    learning_rate=best_rsvd_params['learning_rate'],
    lambda_reg=best_rsvd_params['lambda_reg'],
    epochs=100
)
```

Program pada Tabel 4.33 memuat hyperparameter RSVD terbaik dari file tuning_progress_rsvd.json, kemudian menginisialisasi objek RSVD dengan nilai *n_factors*, *learning_rate*, dan *lambda_reg* dari hasil *tuning*. Jumlah

epoch ditetapkan sebesar 100 untuk memastikan proses dekomposisi matriks berjalan hingga konvergen pada pelatihan final.

Tabel 4.34. Cell *Training* Model RSVD Pada Training.ipynb

```
# Train dengan data asli
final_rsvd.fit(train_data)

# Mengambil loss dari evaluasi akhir array
final_total_loss = final_rsvd.loss_history[-1]
```

Program pada **Tabel 4.34** menjalankan pelatihan RSVD dengan memanggil metode `fit` pada `train_data`. Setelah pelatihan selesai, nilai *loss* pada iterasi terakhir diambil dari `loss_history[-1]` dan ditampilkan sebagai informasi konvergensi akhir model.

Tabel 4.35. Cell Menyimpan Bobot dan *Loss History* RSVD Pada Training.ipynb

```
np.save(os.path.join(save_dir, 'final_mu.npy'),
np.array(final_rsvd.mu))
np.save(os.path.join(save_dir, 'final_b_u.npy'),
final_rsvd.b_u)
np.save(os.path.join(save_dir, 'final_b_i.npy'),
final_rsvd.b_i)
np.save(os.path.join(save_dir, 'best_U.npy'),
final_rsvd.U)
np.save(os.path.join(save_dir, 'best_Sigma.npy'),
final_rsvd.Sigma)
np.save(os.path.join(save_dir, 'best_V.npy'),
final_rsvd.V)

np.save(os.path.join(loss_history_dir,
'final_rsvd_loss_history.npy'), final_rsvd.loss_history)
```

Program pada **Tabel 4.35** ini menyimpan hasil pelatihan RSVD. Komponen dekomposisi matriks disimpan secara terpisah dalam format `.npy`. Riwayat *loss* pelatihan disimpan dalam file `final_rsvd_loss_history.npy` ke direktori `LossHistory` untuk keperluan visualisasi.

4.1.5 Program Cross Validation

Notebook CrossValidation.ipynb mengimplementasikan proses validasi silang 10-fold untuk mengevaluasi konsistensi dan kemampuan generalisasi model VAE, RSVD, dan *hybrid ensemble* secara menyeluruh. *Notebook* ini terdiri dari sebelas seksi utama, yaitu impor *library*, konfigurasi direktori, pemuatan data dan hyperparameter, konfigurasi parameter CV, pembentukan *fold*, definisi fungsi-fungsi pembantu, eksekusi loop CV, perhitungan ringkasan statistik beserta penyimpanan hasil, dan uji signifikansi statistik *paired t-test*. Implementasi pada *notebook* ini ditampilkan pada Tabel 4.36 hingga Tabel 4.48.

1. Import Library

Tabel 4.36. Cell *Import Library* Pada CrossValidation.ipynb

```
import os
import sys
import gc
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
from datetime import datetime
```

Program pada Tabel 4.36 mengimpor seluruh *library* yang dibutuhkan selama proses validasi silang. Selain *library* standar yang juga digunakan di *notebook* sebelumnya, terdapat dua tambahan yang spesifik untuk kebutuhan CV yaitu modul *gc* (*garbage collector*) untuk pembersihan memori secara eksplisit antar *fold*, dan *matplotlib.ticker* untuk pemformatan sumbu pada grafik visualisasi hasil CV.

2. Konfigurasi Direktori

Tabel 4.37. Cell Konfigurasi Direktori Pada CrossValidation.ipynb

```
# Menambahkan folder Scripts ke path agar model.py bisa
diimport
scripts_path = os.path.abspath(os.path.join('.',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)
```



```
# Path folder data
data_dir = os.path.abspath(os.path.join '..',
'Data', 'TrainTest'))
hyperparameter_dir = os.path.abspath(os.path.join '..',
'Data', 'HyperParameters'))
cv_output_dir = os.path.abspath(os.path.join '..',
'Data', 'CrossValidation'))

# Buat folder CrossValidation jika belum ada
os.makedirs(cv_output_dir, exist_ok=True)
```

Program pada **Tabel 4.37** mendefinisikan seluruh jalur direktori yang dibutuhkan. Variabel `scripts_path` ditambahkan ke `sys.path` agar `model.py` dapat diimpor. Variabel `data_dir` mengarah ke direktori data latih, `hyperparameter_dir` ke direktori hasil *tuning*, dan `cv_output_dir` ke direktori keluaran hasil CV. Direktori `cv_output_dir` dibuat secara otomatis menggunakan `os.makedirs` dengan parameter `exist_ok=True` apabila belum tersedia.

3. Load Data & Hyperparameter

Tabel 4.38 menampilkan cell impor kelas model, pemuatan data, dan pemuatan hyperparameter yang digabungkan sebagai satu blok persiapan.

Tabel 4.38. Cell *Import Model, Load Data dan Hyperparameter* Pada *CrossValidation.ipynb*

```
from model import Encoder, Decoder, VAE, RSVD,
KLANnealingCallback

# Load matriks rating yang sudah dinormalisasi (0-1),
belum di-split
normalized_matrix_path = os.path.join(data_dir,
'normalized_matrix.npy')

try:
    normalized_matrix =
np.load(normalized_matrix_path).astype(np.float32)
    num_users, num_items = normalized_matrix.shape
```

```

except FileNotFoundError:
    print("[ERROR] File 'normalized_matrix.npy' tidak
ditemukan.")

# Load hyperparameter terbaik VAE
vae_progress_path = os.path.join(hyperparameter_dir,
'tuning_progress_vae.json')
with open(vae_progress_path, 'r') as f:
    best_vae_params = json.load(f)['best_params']

# Load hyperparameter terbaik RSVD
rsvd_progress_path = os.path.join(hyperparameter_dir,
'tuning_progress_rsvd.json')
with open(rsvd_progress_path, 'r') as f:
    best_rsvd_params = json.load(f)['best_params']

```

Cell ini menjalankan tiga persiapan sekaligus. Pertama, kelas-kelas model diimpor dari `model.py`. Kedua, matriks *rating* yang telah dinormalisasi dimuat dari file `normalized_matrix.npy`. Dimensi matriks diekstrak sebagai `num_users` dan `num_items`. Ketiga, hyperparameter terbaik VAE dan RSVD dimuat dari masing-masing file JSON hasil *tuning* tanpa melakukan *tuning* ulang di setiap *fold*, karena hal tersebut akan memakan waktu komputasi yang tidak proporsional.

4. Konfigurasi Cross-Validation

Tabel 4.39. Cell Konfigurasi *Cross-Validation* Pada `CrossValidation.ipynb`

```

# Jumlah fold untuk K-Fold Cross Validation
N_FOLDS = 10

# Epoch untuk setiap fold (lebih sedikit dari Training
final karena tujuannya konsistensi)
CV_VAE_EPOCHS = 50
CV_RSVD_EPOCHS = 30

# Skala rating asli untuk denormalisasi hasil prediksi
MAX_RATING = 5.0

# Seed acak untuk memastikan hasil dapat direplikasi

```

```
RANDOM_SEED = 42
```

Program pada **Tabel 4.39** mendefinisikan seluruh konstanta konfigurasi CV dalam satu lokasi terpusat. Variabel `N_FOLDS` ditetapkan sebesar 10 sesuai skema *10-fold cross-validation*. Jumlah *epoch* per *fold* untuk VAE (`CV_VAE_EPOCHS = 50`) dan RSVD (`CV_RSVD_EPOCHS = 30`) ditetapkan lebih kecil dibandingkan pelatihan final untuk menjaga efisiensi komputasi. Variabel `MAX_RATING` bernilai 5.0 digunakan untuk mendenormalisasi prediksi kembali ke skala rating asli (1–5). Variabel `RANDOM_SEED = 42` digunakan untuk memastikan hasil pembagian *fold* dapat direplikasi.

5. Pembentukan *Fold*

Tabel 4.40 menampilkan cell ekstraksi koordinat rating dan pembagian indeks ke dalam *fold* yang digabungkan sebagai satu blok pembentukan *fold*.

Tabel 4.40. Cell Ekstraksi Koordinat Rating dan Pembagian *Fold* Pada `CrossValidation.ipynb`

```
# Ambil koordinat (baris, kolom) dari semua rating yang
ada nilainya (> 0)
all_rows, all_cols = np.nonzero(normalized_matrix)

total_ratings = len(all_rows)

# Set seed untuk reproduksibilitas
np.random.seed(RANDOM_SEED)

# Acak urutan semua indeks rating
shuffled_indices = np.random.permutation(total_ratings)

# Bagi indeks yang sudah diacak menjadi N_FOLDS bagian
yang (hampir) sama besar
fold_index_groups = np.array_split(shuffled_indices,
N_FOLDS)
```

Cell ini membangun struktur pembagian *fold* dari matriks *rating* penuh. Fungsi `np.nonzero` digunakan untuk mengekstrak koordinat baris dan kolom dari seluruh 100.000 entri *rating* yang tersedia. Seluruh indeks tersebut kemudian diacak menggunakan `np.random.permutation` dengan *seed* tetap

untuk memastikan reproduksibilitas, lalu dibagi menjadi 10 kelompok yang hampir sama besar menggunakan `np.array_split`.

6. Fungsi Pembantu (*Helper Functions*)

Seksi ini mendefinisikan lima fungsi pembantu yang digunakan di dalam loop CV, ditampilkan pada Tabel 4.41 hingga Tabel 4.45.

Tabel 4.41. Cell Fungsi `create_fold_matrices` Pada `CrossValidation.ipynb`

```
def create_fold_matrices(normalized_matrix, all_rows,
all_cols, test_fold_indices, val_ratio=0.1,
random_seed=42):
    # LANGKAH 1: Pisahkan TEST dari keseluruhan data
    test_rows = all_rows[test_fold_indices]
    test_cols = all_cols[test_fold_indices]

    test_matrix = np.zeros(normalized_matrix.shape,
dtype=np.float32)
    test_matrix[test_rows, test_cols] =
normalized_matrix[test_rows, test_cols]

    train_full_matrix = normalized_matrix.copy()
    train_full_matrix[test_rows, test_cols] = 0.0

    # LANGKAH 2: Pisahkan VAL_INNER dari TRAIN dengan
hold-out
    train_rows, train_cols = np.nonzero(train_full_matrix)
    n_train = len(train_rows)
    n_val = int(n_train * val_ratio)

    rng = np.random.default_rng(random_seed)
    val_chosen = rng.choice(n_train, size=n_val,
replace=False)
    val_inner_rows = train_rows[val_chosen]
    val_inner_cols = train_cols[val_chosen]

    val_inner_values = train_full_matrix[val_inner_rows,
val_inner_cols].copy()
    val_inner_indices = (val_inner_rows, val_inner_cols)
```

```

train_inner_matrix = train_full_matrix.copy()
train_inner_matrix[val_inner_rows, val_inner_cols] =
0.0

return train_inner_matrix, val_inner_indices,
val_inner_values, test_matrix, (test_rows, test_cols)

```

Fungsi `create_fold_matrices` pada **Tabel 4.41** bertugas membentuk tiga partisi data untuk setiap *fold* yaitu `train_inner_matrix`, `val_inner`, dan `test_matrix`. Fungsi ini bekerja dalam dua langkah. Pertama, rating pada *fold* yang sedang diproses dipisahkan sebagai *test set* dengan mengisi `test_matrix` dari koordinat `test_fold_indices`, sementara rating tersebut dihapus dari salinan matriks penuh untuk membentuk `train_full_matrix`. Kedua, sebesar 10% dari `train_full_matrix` dipisahkan secara acak sebagai `val_inner` menggunakan `np.random.default_rng` dengan *seed* tetap yang nanti akan digunakan untuk mencari bobot *ensemble* alpha tanpa menyentuh *test set*. Sisa rating yang tersisa membentuk `train_inner_matrix` yang digunakan untuk melatih VAE dan RSVD. Struktur partisi per *fold* adalah *test* (10%), *val_inner* (9%), dan *train_inner* (81%).

Tabel 4.42. Cell Fungsi `train_vae_fold` Pada `CrossValidation.ipynb`

```

def train_vae_fold(train_matrix, vae_params, num_items,
epochs, kl_annealing_epochs=15):
# Bersihkan sesi TensorFlow sebelum membuat model baru
    tf.keras.backend.clear_session()

    # Ubah data ke TensorFlow constant agar tidak ada
    duplikasi memori
    train_tf = tf.constant(train_matrix, dtype=tf.float32)

    # Bangun arsitektur VAE dengan hyperparameter terbaik
    encoder = Encoder(
        hidden_dims=vae_params['hidden_dims'],
        latent_dim=vae_params['latent_dim'],
        dropout_rate=vae_params['dropout_rate']
    )
    decoder = Decoder(
        hidden_dims=vae_params['hidden_dims'][:-1],
        output_dim=num_items
    )

```

```

    )

    # Inisialisasi VAE dengan beta=0.0 agar KL annealing
    mulai dari nol
    vae_model = VAE(encoder, decoder, beta=0.0)

    # Inisialisasi bobot dengan melewati 1 baris data
    _ = vae_model(train_tf[:1])

    # Kompilasi dengan optimizer Adam
    optimizer =
tf.keras.optimizers.Adam(learning_rate=vae_params['learnin
g_rate'])
    vae_model.compile(optimizer=optimizer,
run_eagerly=True)

    # Callback KL annealing: naikan beta dari 0 ke
vae_params['beta'] secara linear
    kl_annealing = KLANnealingCallback(
        beta_target=vae_params['beta'],
        annealing_epochs=kl_annealing_epochs
    )

    # Latih model dengan KL annealing (verbose=0 agar
    output tidak terlalu panjang)
    vae_model.fit(
        train_tf, train_tf,
        epochs=epochs,
        batch_size=vae_params['batch_size'],
        callbacks=[kl_annealing],
        verbose=0
    )

    # Ekstrak ruang laten (z_mean) lalu rekonstruksi
    dengan decoder
    z_mean, _ = vae_model.encoder.predict(train_tf,
    verbose=0)

```

```

    pred_matrix = vae_model.decoder.predict(z_mean,
verbose=0)

    # Bersihkan semua objek model dari memori
    del encoder, decoder, vae_model, optimizer, train_tf,
z_mean, _, kl_annealing
    gc.collect()
    tf.keras.backend.clear_session()

    return pred_matrix

```

Fungsi `train_vae_fold` pada **Tabel 4.42** melatih satu instans VAE untuk satu *fold* dan mengembalikan matriks prediksi rekonstruksi penuh berukuran 943×1682 . Fungsi ini memanggil `tf.keras.backend.clear_session()` di awal dan akhir eksekusi untuk membersihkan grafik komputasi TensorFlow dan mencegah *memory leak* antar *fold*. Arsitektur VAE dibangun ulang dari nol menggunakan hyperparameter terbaik, dilatih dengan `KLAnnealingCallback` tanpa `EarlyStopping` untuk menjaga jumlah *epoch* tetap terkontrol. Prediksi dihasilkan melalui dua langkah yaitu `encoder.predict` menghasilkan `z_mean` dari ruang *laten*, kemudian `decoder.predict` merekonstruksi prediksi *rating* dari `z_mean` tersebut. Seluruh objek model dihapus secara eksplisit menggunakan `del` dan `gc.collect()` setelah prediksi selesai diambil.

Tabel 4.43. Cell Fungsi `train_rsvd_fold` Pada `CrossValidation.ipynb`

```

def train_rsvd_fold(train_matrix, rsvd_params, epochs):
    # Inisialisasi model RSVD dengan hyperparameter
    terbaik
    rsvd_model = RSVD(
        n_factors=rsvd_params['n_factors'],
        learning_rate=rsvd_params['learning_rate'],
        lambda_reg=rsvd_params['lambda_reg'],
        epochs=epochs
    )

    # Latih model dengan data training fold ini
    rsvd_model.fit(train_matrix)

    # Rekonstruksi prediksi penuh: bias + interaksi laten

```

```

        bias_matrix = float(rsvd_model.mu) + rsvd_model.b_u[:,
np.newaxis] + rsvd_model.b_i[np.newaxis, :]

        latent_matrix = np.dot(np.dot(rsvd_model.U,
rsvd_model.Sigma), rsvd_model.V.T)
        pred_matrix = bias_matrix + latent_matrix

        # Bersihkan objek model dari memori
        del rsvd_model, bias_matrix, latent_matrix
        gc.collect()

    return pred_matrix

```

Fungsi `train_rsvd_fold` pada **Tabel 4.43** melatih satu instans RSVD untuk satu *fold* dan mengembalikan matriks prediksi penuh berukuran 943×1682 . Setelah pelatihan selesai melalui metode `fit`, prediksi rating direkonstruksi dengan menjumlahkan komponen *bias* yang terdiri dari *global bias*, *user bias*, dan *item bias* dengan komponen interaksi *laten* yang diperoleh dari perkalian matriks *U*, *Sigma*, dan *V.T*. Seluruh objek model dan matriks antara dihapus menggunakan `del` dan `gc.collect()` setelah prediksi selesai.

Tabel 4.44. Cell Fungsi `find_best_ensemble_alpha` Pada `CrossValidation.ipynb`

```

def find_best_ensemble_alpha(pred_vae_full,
pred_rsvd_full, val_inner_indices, val_inner_values):
    best_alpha = 0.0
    best_val_rmse = float('inf')

    # Ambil prediksi di koordinat val_inner lalu
    denormalisasi ke skala 1-5
    val_pred_vae =
np.clip(pred_vae_full[val_inner_indices] * MAX_RATING,
1.0, MAX_RATING)
    val_pred_rsvd =
np.clip(pred_rsvd_full[val_inner_indices] * MAX_RATING,
1.0, MAX_RATING)
    val_actual = val_inner_values * MAX_RATING

```



```

# Coba setiap nilai alpha dari 0% hingga 100% dengan
langkah 1%
for alpha in np.linspace(0, 1, 101):
    beta = 1.0 - alpha
    pred_ensemble = np.clip((alpha * val_pred_vae) +
(beta * val_pred_rsvd), 1.0, MAX_RATING)
    rmse = np.sqrt(np.mean(np.square(val_actual -
pred_ensemble)))

    if rmse < best_val_rmse:
        best_val_rmse = rmse
        best_alpha = alpha

return best_alpha, best_val_rmse

```

Fungsi `find_best_ensemble_alpha` pada [Tabel 4.44](#) mencari bobot α optimal untuk *weighted average ensemble* menggunakan data `val_inner`, bukan *test set*, sehingga evaluasi akhir di *test set* tetap tidak bias. Fungsi ini mengiterasi 101 nilai α dari 0,00 hingga 1,00 dengan langkah 0,01 menggunakan `np.linspace`. Pada setiap iterasi, prediksi *ensemble* dihitung sebagai $(\alpha \times \text{VAE}) + ((1 - \alpha) \times \text{RSVD})$ dan RMSE-nya dievaluasi terhadap `val_inner`. Nilai α yang menghasilkan RMSE terkecil di `val_inner` dipilih sebagai bobot terbaik untuk *fold* tersebut.

Tabel 4.45. Cell Fungsi `calculate_metrics` Pada `CrossValidation.ipynb`

```

def calculate_metrics(y_true, y_pred):
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae

```

Fungsi `calculate_metrics` pada [Tabel 4.45](#) menghitung tiga metrik evaluasi regresi sekaligus dari pasangan nilai aktual `y_true` dan nilai prediksi `y_pred`. MSE dihitung sebagai rata-rata kuadrat selisih, RMSE sebagai akar dari MSE, dan MAE sebagai rata-rata nilai absolut selisih. Ketiga metrik dikembalikan sekaligus dalam satu pemanggilan fungsi untuk efisiensi.

7. Loop K-Fold Cross-Validation

Tabel 4.46. Cell *Loop Utama Cross-Validation* Pada `CrossValidation.ipynb`

```

cv_results = { 'fold': [], 'vae_mse': [], 'vae_rmse': [],
'vae_mae': [], 'rsvd_mse': [], 'rsvd_rmse': [],
'rsvd_mae': [], 'hybrid_mse': [], 'hybrid_rmse': [],
'hybrid_mae': [], 'best_alpha': [], 'best_val_rmse': [],
'test_size': []}

for fold_idx, test_fold_indices in
enumerate(fold_index_groups):
    fold_num = fold_idx + 1
    # LANGKAH 1: Buat matriks train_inner, val_inner, dan
test
    train_inner_matrix, val_inner_indices,
val_inner_values, test_matrix, test_indices =
create_fold_matrices(
    normalized_matrix, all_rows, all_cols,
test_fold_indices,
    val_ratio=0.1, random_seed=RANDOM_SEED
    )

    # Ambil nilai aktual dari test set lalu denormalisasi
actual_values_norm = test_matrix[test_indices]
actual_values = actual_values_norm * MAX_RATING

    # LANGKAH 2: Training dan Prediksi VAE
pred_vae_full = train_vae_fold(
    train_inner_matrix, best_vae_params, num_items,
CV_VAE_EPOCHS
    )

    # LANGKAH 3: Training dan Prediksi RSVD
pred_rsvd_full = train_rsvd_fold(
    train_inner_matrix, best_rsvd_params,
CV_RSVD_EPOCHS
    )

    # LANGKAH 4: Cari alpha ensemble terbaik menggunakan
VAL_INNER
    best_alpha, best_val_rmse = find_best_ensemble_alpha(

```

```

        pred_vae_full, pred_rsvd_full,
        val_inner_indices, val_inner_values
    )
    best_beta = 1.0 - best_alpha

    # LANGKAH 5: Evaluasi di TEST SET menggunakan alpha
    dari val_inner
    pred_vae_test = np.clip(pred_vae_full[test_indices] *
MAX_RATING, 1.0, MAX_RATING)
    pred_rsvd_test = np.clip(pred_rsvd_full[test_indices]
* MAX_RATING, 1.0, MAX_RATING)
    pred_hybrid_test = np.clip(
        (best_alpha * pred_vae_test) + (best_beta *
pred_rsvd_test),
        1.0, MAX_RATING
    )

    # Hitung metrik evaluasi
    mse_v, rmse_v, mae_v =
calculate_metrics(actual_values, pred_vae_test)
    mse_r, rmse_r, mae_r =
calculate_metrics(actual_values, pred_rsvd_test)
    mse_h, rmse_h, mae_h =
calculate_metrics(actual_values, pred_hybrid_test)

    # LANGKAH 6: Simpan hasil fold ke dictionary
    cv_results['fold'].append(fold_num)
    cv_results['test_size'].append(n_test)
    cv_results['best_alpha'].append(float(best_alpha))
    cv_results['best_val_rmse'].append(float(best_val_rmse
))

    cv_results['vae_mse'].append(float(mse_v))
    cv_results['vae_rmse'].append(float(rmse_v))
    cv_results['vae_mae'].append(float(mae_v))

    cv_results['rsvd_mse'].append(float(mse_r))

```

```

cv_results['rsvd_rmse'].append(float(rmse_r))
cv_results['rsvd_mae'].append(float(mae_r))

cv_results['hybrid_mse'].append(float(mse_h))
cv_results['hybrid_rmse'].append(float(rmse_h))
cv_results['hybrid_mae'].append(float(mae_h))

# Bersihkan memori sebelum fold berikutnya
del train_inner_matrix, test_matrix
del val_inner_indices, val_inner_values
del actual_values, actual_values_norm
del pred_vae_full, pred_rsvd_full
del pred_vae_test, pred_rsvd_test, pred_hybrid_test
gc.collect()

```

Program pada **Tabel 4.46** mengeksekusi loop utama *10-fold cross-validation* dalam enam langkah berurutan untuk setiap *fold*. Langkah pertama memanggil `create_fold_matrices` untuk membentuk partisi `train_inner`, `val_inner`, dan `test` pada *fold* yang sedang berjalan. Langkah kedua dan ketiga melatih VAE dan RSVD secara terpisah menggunakan `train_inner_matrix` melalui fungsi `train_vae_fold` dan `train_rsvd_fold`. Langkah keempat mencari bobot *alpha ensemble* terbaik pada `val_inner` menggunakan `find_best_ensemble_alpha` tanpa menyentuh *test set*. Langkah kelima menerapkan *alpha* tersebut untuk menghasilkan prediksi *hybrid* pada *test set*, lalu menghitung MSE, RMSE, dan MAE ketiga model menggunakan `calculate_metrics`. Langkah keenam menyimpan seluruh hasil ke *dictionary* `cv_results`, diikuti pembersihan memori secara eksplisit menggunakan `del` dan `gc.collect()` sebelum *fold* berikutnya dimulai.

8. Perhitungan Ringkasan dan Penyimpanan Hasil

Tabel 4.47. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada `CrossValidation.ipynb`

```

# Metrik yang ingin diringkaskan
metric_keys = [
    'vae_mse', 'vae_rmse', 'vae_mae',
    'rsvd_mse', 'rsvd_rmse', 'rsvd_mae',
    'hybrid_mse', 'hybrid_rmse', 'hybrid_mae'
]

```

```

# Hitung mean dan std dev untuk setiap metrik
cv_summary = {}
for key in metric_keys:
    values = np.array(cv_results[key])
    cv_summary[key] = {
        'mean' : float(np.mean(values)),
        'std' : float(np.std(values)),
        'min' : float(np.min(values)),
        'max' : float(np.max(values))
    }

timestamp = datetime.now().strftime('%Y%m%d_%H%M')
cv_run_dir = os.path.join(cv_output_dir,
f'cv_{timestamp}')
os.makedirs(cv_run_dir, exist_ok=True)

final_cv_output = {
    'timestamp': timestamp,
    'configuration': {
        'n_folds': N_FOLDS,
        'cv_vae_epochs': CV_VAE_EPOCHS,
        'cv_rsvd_epochs': CV_RSVD_EPOCHS,
        'random_seed': RANDOM_SEED,
        'vae_params': best_vae_params,
        'rsvd_params': best_rsvd_params
    },
    'per_fold_results': cv_results,
    'summary': cv_summary
}

cv_json_path = os.path.join(cv_run_dir,
f'cv_results_{timestamp}.json')
with open(cv_json_path, 'w') as f:
    json.dump(final_cv_output, f, indent=4)

```

Program pada **Tabel 4.47** menjalankan dua operasi pasca-pemrosesan. Pertama, ringkasan statistik dihitung untuk setiap metrik dari seluruh 10 *fold*, meliputi nilai *mean*, *standard deviation*, *minimum*, dan *maksimum*,

yang disimpan dalam *dictionary* `cv_summary`. Kedua, seluruh hasil CV digabungkan ke dalam satu *dictionary* `final_cv_output` dan disimpan ke file JSON dalam subfolder bertanda waktu (format `cv_YYYYMMDD_HHMM`) di dalam direktori `CrossValidation`. Penamaan berbasis *timestamp* memastikan setiap sesi CV tersimpan rapi tanpa menimpa hasil sebelumnya.

9. Uji Signifikansi Statistik *Paired t-Test*

Tabel 4.48. Cell Perhitungan Ringkasan dan Penyimpanan Hasil Pada `CrossValidation.ipynb`

```
from scipy import stats

cv_output_dir = os.path.abspath(os.path.join('..', 'Data',
'CrossValidation'))
CV_RESULT_FILE =
'cv_20260402_2042/cv_results_20260402_2042.json'
cv_json_path = os.path.join(cv_output_dir, CV_RESULT_FILE)

try:
    with open(cv_json_path, 'r') as f:
        cv_data = json.load(f)
except FileNotFoundError:
    print(f"[ERROR] File tidak ditemukan: {cv_json_path}")

# Ekstrak RMSE dan MAE per fold untuk setiap model
per_fold = cv_data['per_fold_results']
N_FOLDS = cv_data['configuration']['n_folds']
rmse_vae = np.array(per_fold['vae_rmse'])
rmse_rsvd = np.array(per_fold['rsvd_rmse'])
rmse_hybrid = np.array(per_fold['hybrid_rmse'])
mae_vae = np.array(per_fold['vae_mae'])
mae_rsvd = np.array(per_fold['rsvd_mae'])
mae_hybrid = np.array(per_fold['hybrid_mae'])

# Paired t-test untuk RMSE
t_rmse_h_r, p_rmse_h_r = stats.ttest_rel(rmse_hybrid,
rmse_rsvd)
t_rmse_h_v, p_rmse_h_v = stats.ttest_rel(rmse_hybrid,
rmse_vae)
```

```

t_rmse_r_v, p_rmse_r_v =
stats.ttest_rel(rmse_rsvd,    rmse_vae)

# Paired t-test untuk MAE
t_mae_h_r, p_mae_h_r = stats.ttest_rel(mae_hybrid,
mae_rsvd)
t_mae_h_v, p_mae_h_v = stats.ttest_rel(mae_hybrid,
mae_vae)
t_mae_r_v, p_mae_r_v =
stats.ttest_rel(mae_rsvd,    mae_vae)

alpha = 0.05
ttest_results = {
    'alpha': alpha,
    'n_folds': N_FOLDS,
    'rmse': {
        'hybrid_vs_rsvd': {'t_statistic':
float(t_rmse_h_r), 'p_value': float(p_rmse_h_r),
'significant': bool(p_rmse_h_r < alpha)},
        'hybrid_vs_vae': {'t_statistic':
float(t_rmse_h_v), 'p_value': float(p_rmse_h_v),
'significant': bool(p_rmse_h_v < alpha)},
        'rsvd_vs_vae': {'t_statistic': float(t_rmse_r_v),
'p_value': float(p_rmse_r_v), 'significant':
bool(p_rmse_r_v < alpha)},
    },
    'mae': {
        'hybrid_vs_rsvd': {'t_statistic':
float(t_mae_h_r), 'p_value': float(p_mae_h_r),
'significant': bool(p_mae_h_r < alpha)},
        'hybrid_vs_vae': {'t_statistic': float(t_mae_h_v),
'p_value': float(p_mae_h_v), 'significant': bool(p_mae_h_v
< alpha)},
        'rsvd_vs_vae': {'t_statistic': float(t_mae_r_v),
'p_value': float(p_mae_r_v), 'significant': bool(p_mae_r_v
< alpha)},
    },

```

```

}

cv_data['paired_ttest'] = ttest_results
with open(cv_json_path, 'w') as f:
    json.dump(cv_data, f, indent=4)

```

Program pada **Tabel 4.48** melaksanakan uji *paired t-test* untuk membuktikan bahwa perbedaan performa antar model tidak terjadi secara kebetulan. Uji dilakukan terhadap dua metrik, yaitu RMSE dan MAE, dengan tiga pasangan perbandingan: *Hybrid* vs RSVD, *Hybrid* vs VAE, dan RSVD vs VAE. Fungsi `stats.ttest_rel` dari *library* *scipy* digunakan karena setiap pasangan pengamatan berasal dari *fold* yang sama, sehingga data bersifat berpasangan (*paired*). Hipotesis nol (H_0) menyatakan tidak ada perbedaan signifikan antara dua model, dan H_0 ditolak apabila *p-value* < 0,05. Hasil uji disimpan kembali ke file JSON hasil CV yang sama di bawah *key* `paired_ttest`.

4.1.6 Program Testing

Notebook `Testing.ipynb` mengimplementasikan proses pengujian akhir model VAE, RSVD, dan *hybrid ensemble* pada *hold-out test set*. *Notebook* ini terdiri dari enam seksi utama, yaitu impor *library*, konfigurasi direktori, pemuatan data dan model, prediksi ketiga model, pencarian dan penerapan bobot *ensemble*, serta evaluasi dan penyimpanan hasil. Implementasi pada *notebook* ini ditampilkan pada **Tabel 4.49 hingga Tabel 4.60**.

1. Import Library

Tabel 4.49. Cell *Import Library* Pada `Testing.ipynb`

```

import os
import sys
import json
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from datetime import datetime

```


Program pada **Tabel 4.49** mengimpor seluruh *library* yang dibutuhkan selama proses pengujian. *Library* `os` dan `sys` digunakan untuk manajemen jalur sistem, `json` untuk pemuatan hyperparameter dan penyimpanan hasil evaluasi, `numpy` untuk operasi numerik, `tensorflow` untuk inferensi VAE, `matplotlib.pyplot` untuk visualisasi kurva pencarian alpha, serta `datetime` untuk pemberian *timestamp* pada file keluaran.

2. Konfigurasi Direktori

Tabel 4.50. Cell Konfigurasi Direktori Pada `Testing.ipynb`

```
scripts_path = os.path.abspath(os.path.join '..',
'Scripts'))
if scripts_path not in sys.path:
    sys.path.append(scripts_path)

data_dir = os.path.abspath(os.path.join '..', 'Data',
'TrainTest'))
save_dir = os.path.abspath(os.path.join '..', 'Data',
'SavedModels'))
hyperparameters_dir = os.path.abspath(os.path.join '..',
'Data', 'HyperParameters'))
visualization_dir = os.path.abspath(os.path.join '..',
'Data', 'Visualization'))
results_dir = os.path.abspath(os.path.join '..', 'Data',
'EvaluationResults'))
```

Program pada **Tabel 4.50** mendefinisikan lima jalur direktori yang digunakan selama pengujian. Variabel `scripts_path` ditambahkan ke `sys.path` agar `model.py` dapat diimpor. Variabel `data_dir` mengarah ke direktori data latih dan uji, `save_dir` ke direktori bobot model terlatih, `hyperparameters_dir` ke direktori hasil *tuning*, `visualization_dir` ke direktori penyimpanan gambar, dan `results_dir` ke direktori penyimpanan hasil evaluasi akhir.

3. Load Data & Model

Tabel 4.51. Cell Load Data dan Model Pada `Testing.ipynb`

```
# Load data training (70% dari total rating)
# Dipakai sebagai input encoder VAE untuk mengekstrak
representasi laten
```

```

train_data = np.load(os.path.join(data_dir,
"train_data.npy")).astype(np.float32)
num_users, num_items = train_data.shape

# Load data validasi (10% dari total rating)
# Dipakai untuk mencari bobot ensemble optimal (alpha)
tanpa melihat test set
MAX_RATING = 5.0

val_data = np.load(os.path.join(data_dir,
"val_data.npy")).astype(np.float32)
val_indices = np.where(val_data > 0)

# Denormalisasi nilai aktual validasi ke skala rating asli
(1-5)
actual_val_values = val_data[val_indices] * MAX_RATING

# Load data test (20% dari total rating)
# Hanya disentuh satu kali untuk pelaporan performa akhir
model
test_data = np.load(os.path.join(data_dir,
"test_data.npy")).astype(np.float32)
test_indices = np.where(test_data > 0)

# Denormalisasi nilai aktual test ke skala rating asli (1-
5)
actual_test_values = test_data[test_indices] * MAX_RATING

from model import Encoder, Decoder, VAE

```

Program pada **Tabel 4.51** memuat tiga partisi data sekaligus mengimpor kelas model. Data latih dimuat dari `train_data.npy` dan digunakan sebagai input *encoder* VAE pada tahap inferensi untuk mengekstrak representasi *laten* seluruh pengguna. Data validasi dimuat dari `val_data.npy` dan digunakan semata-mata untuk mencari bobot *alpha ensemble* yang optimal tanpa menyentuh *test set*, sehingga evaluasi akhir tetap tidak bias. Data uji dimuat dari `test_data.npy` dan hanya digunakan satu kali pada tahap evaluasi akhir. Nilai aktual dari data validasi dan data uji didenormalisasi ke skala

rating 1–5 dengan mengalikan $\text{MAX_RATING} = 5.0$. Koordinat entri *rating* yang tersedia pada kedua partisi tersebut diekstrak menggunakan `np.where`.

4. Konstruksi dan *Load* Bobot Model

Seksi ini terdiri dari dua tahap yang ditampilkan pada **Tabel 4.52 dan Tabel 4.53**.

Tabel 4.52. Cell Konstruksi dan *Load* Bobot VAE Pada *Testing.ipynb*

```
# Load hyperparameter terbaik
vae_progress_file = os.path.join(hyperparameters_dir,
'tuning_progress_vae.json')
with open(vae_progress_file, 'r') as f:
    best_vae_params = json.load(f)['best_params']

# Membangun Arsitektur VAE
encoder =
Encoder(hidden_dims=best_vae_params['hidden_dims'],
latent_dim=best_vae_params['latent_dim'],
dropout_rate=best_vae_params['dropout_rate'])
decoder =
Decoder(hidden_dims=best_vae_params['hidden_dims'][:-1],
output_dim=num_items)
eval_vae = VAE(encoder, decoder,
beta=best_vae_params['beta'])

# Menyuntikkan Bobot ke VAE
_ = eval_vae(train_data[:1])
eval_vae.load_weights(os.path.join(save_dir,
'trained_best_vae_weights.weights.h5'))
```

Program pada **Tabel 4.52** membangun ulang arsitektur VAE dari hyperparameter terbaik yang dimuat dari `tuning_progress_vae.json`, kemudian memuat bobot terlatih dari file `.h5` yang dihasilkan oleh *Training.ipynb*. *Forward pass* awal pada satu sampel data (`train_data[:1]`) dilakukan terlebih dahulu untuk menginisialisasi bobot model sebelum `load_weights` dipanggil, karena TensorFlow memerlukan bobot model untuk dibangun terlebih dahulu sebelum dapat dimuat dari file.

Tabel 4.53. Cell *Load* Komponen Bobot RSVD Pada Testing.ipynb

```
# Memuat komponen Bias
mu = np.load(os.path.join(save_dir, 'final_mu.npy'))
b_u = np.load(os.path.join(save_dir, 'final_b_u.npy'))
b_i = np.load(os.path.join(save_dir, 'final_b_i.npy'))

# Memuat komponen Laten
U = np.load(os.path.join(save_dir, 'best_U.npy'))
Sigma = np.load(os.path.join(save_dir, 'best_Sigma.npy'))
V = np.load(os.path.join(save_dir, 'best_V.npy'))
```

Program pada **Tabel 4.53** memuat enam komponen bobot RSVD yang disimpan secara terpisah oleh Training.ipynb dalam format .npy. Komponen *bias* terdiri dari *global bias* *mu*, *user bias* *b_u*, dan *item bias* *b_i*. Komponen *laten* terdiri dari matriks faktor pengguna *U*, matriks nilai singular *Sigma*, dan matriks faktor item *V*. Keenam komponen ini akan digabungkan pada tahap prediksi untuk merekonstruksi matriks *rating* penuh.

5. Prediksi Model

Seksi prediksi terdiri dari dua tahap yang ditampilkan pada **Tabel 4.54 dan Tabel 4.55**.

Tabel 4.54. Cell Prediksi VAE Pada Testing.ipynb

```
# Ubah train data menjadi tensor
train_data_tf = tf.constant(train_data, dtype=tf.float32)

# Ekstrak matriks laten
Z_mean, Z_log_var =
eval_vae.encoder.predict(train_data_tf, verbose=0)

# Prediksi dari Decoder (skala 0 - 1)
pred_vae_norm = eval_vae.decoder.predict(Z_mean,
verbose=0)

# Filter ke koordinat test
pred_vae_test = pred_vae_norm[test_indices]

# Kembalikan ke skala asli
pred_vae_test_denorm = pred_vae_test * MAX_RATING
```

```
# Batasi prediksi VAE agar tidak keluar dari rentang
rating yang valid (1.0 - 5.0)
pred_vae_test_clipped = np.clip(pred_vae_test_denorm, 1.0,
MAX_RATING)
```

Program pada **Tabel 4.54** menghasilkan prediksi *rating* VAE dalam lima langkah berurutan. Pertama, data latih dikonversi ke `tf.constant` untuk diproses oleh TensorFlow. Kedua, `encoder.predict` mengekstrak representasi *laten* `Z_mean` dari seluruh pengguna. Ketiga, `decoder.predict` merekonstruksi prediksi *rating* penuh dalam skala 0–1 dari `Z_mean`. Keempat, prediksi difilter ke koordinat *test set* menggunakan `test_indices`. Kelima, prediksi didenormalisasi ke skala 1–5 dan dibatasi menggunakan `np.clip` agar tidak melebihi rentang *rating* yang valid.

Tabel 4.55. Cell Prediksi RSVD Pada `Testing.ipynb`

```
# Kalkulasi matriks laten
latent_matrix = np.dot(np.dot(U, Sigma), V.T)

# Menghitung matriks bias (rata-rata global + bias
pengguna + bias film)
bias_matrix = float(mu) + b_u[:, np.newaxis] +
b_i[np.newaxis, :]

# Gabungkan bias dan interaksi laten menjadi tebakan penuh
(Skala 0 - 1)
full_rsvd_pred = bias_matrix + latent_matrix

# Filter untuk indeks test data saja
pred_rsvd_test = full_rsvd_pred[test_indices]

# Kembalikan prediksi ke skala asli
pred_rsvd_test_denorm = pred_rsvd_test * MAX_RATING

# Batasi prediksi RSVD agar tidak keluar dari rentang
rating yang valid (1.0 - 5.0)
```

```
pred_rsvd_test_clipped = np.clip(pred_rsvd_test_denorm,  
1.0, MAX_RATING)
```

Program pada **Tabel 4.55** merekonstruksi prediksi *rating* RSVD dari komponen-komponen bobot yang telah dimuat. Matriks interaksi *laten* diperoleh dari perkalian U, Sigma, dan V.T. Matriks *bias* dibangun dengan menjumlahkan *global bias* μ , *user bias* b_u (diperluas secara kolom menggunakan $[:, \text{np.newaxis}]$), dan *item bias* b_i (diperluas secara baris menggunakan $[\text{np.newaxis}, :]$) sehingga menghasilkan matriks berukuran 943×1682 . Prediksi penuh diperoleh dari penjumlahan kedua matriks tersebut, kemudian difilter ke koordinat *test set*, didenormalisasi, dan diclip ke rentang rating yang valid.

6. Pencarian dan Penerapan Bobot *Ensemble*

Seksi ini terdiri dari dua tahap yang ditampilkan pada **Tabel 4.56 dan Tabel 4.57**.

Tabel 4.56. Cell Pencarian Bobot Alpha *Ensemble* Pada Testing.ipynb

```
# Ekstrak prediksi VAE dan RSVD di koordinat validasi  
# Denormalisasi dan clip ke skala 1-5  
pred_vae_val = np.clip(pred_vae_norm[val_indices] *  
MAX_RATING, 1.0, MAX_RATING)  
pred_rsvd_val = np.clip(full_rsvd_pred[val_indices] *  
MAX_RATING, 1.0, MAX_RATING)  
  
best_val_rmse = float("inf")  
best_alpha = 0.0  
best_beta = 0.0  
  
# Menyimpan history untuk visualisasi kurva tuning  
alpha_history = []  
rmse_history = []  
  
# Grid search alpha dari 0.00 hingga 1.00 dengan langkah  
0.01  
# Alpha = proporsi kontribusi VAE, (1 - alpha) = proporsi  
kontribusi RSVD  
for alpha in np.linspace(0, 1, 101):  
    beta = 1.0 - alpha  
    pred_ensemble = np.clip((alpha * pred_vae_val) + (beta  
* pred_rsvd_val), 1.0, MAX_RATING)
```

```

rmse =
np.sqrt(np.mean(np.square(actual_val_values -
pred_ensemble)))

alpha_history.append(alpha)
rmse_history.append(rmse)

if rmse < best_val_rmse:
    best_val_rmse = rmse
    best_alpha = alpha
    best_beta = beta

```

Program pada **Tabel 4.56** Cell ini mencari bobot alpha optimal untuk *weighted average ensemble* menggunakan data validasi. Prediksi VAE dan RSVD diekstrak pada koordinat validasi (`val_indices`), dinormalisasi, dan diclip ke skala 1–5. Selanjutnya, *grid search* dilakukan terhadap 101 nilai alpha dari 0,00 hingga 1,00 dengan langkah 0,01. Pada setiap iterasi, prediksi *ensemble* dihitung sebagai $(\alpha \times \text{VAE}) + ((1 - \alpha) \times \text{RSVD})$ dan RMSE-nya dievaluasi terhadap nilai aktual validasi. Nilai alpha yang menghasilkan RMSE terkecil dipilih sebagai bobot terbaik. Riwayat nilai RMSE per alpha disimpan dalam `rmse_history` untuk keperluan visualisasi.

Tabel 4.57. Cell Penerapan Alpha ke Test Set Pada `Testing.ipynb`

```

# Terapkan alpha terbaik (yang ditemukan di validasi) ke
prediksi test set
pred_vae_test_clipped =
np.clip(pred_vae_norm[test_indices] * MAX_RATING, 1.0,
MAX_RATING)
pred_rsvd_test_clipped =
np.clip(full_rsvd_pred[test_indices] * MAX_RATING, 1.0,
MAX_RATING)

pred_hybrid = (best_alpha * pred_vae_test_clipped) +
(best_beta * pred_rsvd_test_clipped)

# Batasi prediksi agar tidak keluar dari rentang rating
yang valid (1.0 - 5.0)

```

```
pred_hybrid_clipped = np.clip(pred_hybrid, 1.0,
MAX_RATING)
```

Program pada **Tabel 4.57** menerapkan alpha terbaik yang diperoleh dari data validasi ke prediksi *test set*. Prediksi VAE dan RSVD pada koordinat *test set* didenormalisasi dan diclip terlebih dahulu, kemudian digabungkan dengan formula *weighted average* menggunakan *best_alpha* dan *best_beta*. Prediksi *hybrid* hasil penggabungan diclip sekali lagi untuk memastikan seluruh nilai berada dalam rentang 1,0–5,0.

7. Evaluasi dan Penyimpanan Hasil

Seksi evaluasi terdiri dari tiga tahap yang ditampilkan pada **Tabel 4.58**, **Tabel 4.59**, dan **Tabel 4.60**.

Tabel 4.58. Fungsi `calculate_metrics` Pada `Testing.ipynb`

```
# Fungsi helper untuk menghitung MSE, RMSE, dan MAE
sekaligus
def calculate_metrics(y_true, y_pred):
    mse = np.mean(np.square(y_true - y_pred))
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    return mse, rmse, mae
```

Fungsi `calculate_metrics` pada **Tabel 4.58** menghitung tiga metrik evaluasi regresi sekaligus dari pasangan nilai aktual dan prediksi. Implementasi fungsi ini identik dengan yang digunakan pada `CrossValidation.ipynb`, didefinisikan ulang secara lokal agar *notebook* ini dapat dijalankan secara independen tanpa bergantung pada *notebook* lain.

Tabel 4.59. Cell Perhitungan Metrik Pada `Testing.ipynb`

```
# Hitung metrik evaluasi untuk ketiga model di test set
mse_v, rmse_v, mae_v =
calculate_metrics(actual_test_values,
pred_vae_test_clipped) # VAE
mse_r, rmse_r, mae_r =
calculate_metrics(actual_test_values,
pred_rsvd_test_clipped) # RSVD
mse_h, rmse_h, mae_h =
calculate_metrics(actual_test_values, pred_hybrid_clipped)
# Hybrid
```


Program pada **Tabel 4.59** menghitung MSE, RMSE, dan MAE untuk ketiga model pada *test set* menggunakan fungsi `calculate_metrics`. Hasil numerik dari cell ini merupakan performa akhir model.

Tabel 4.60. Cell Penyimpanan Hasil Evaluasi Pada `Testing.ipynb`

```
# Simpan seluruh ringkasan metrik agar bisa dipanggil kembali tanpa perlu re-run
evaluation_results = {
    'timestamp': timestamp,
    'ensemble_weights': {
        'best_alpha_vae': float(best_alpha),
        'best_beta_rsvd': float(best_beta),
        'best_val_rmse': float(best_val_rmse)
    },
    'metrics': {
        'vae_standalone': {'MSE': float(mse_v), 'RMSE': float(rmse_v), 'MAE': float(mae_v)},
        'rsvd_standalone': {'MSE': float(mse_r), 'RMSE': float(rmse_r), 'MAE': float(mae_r)},
        'hybrid_ensemble': {'MSE': float(mse_h), 'RMSE': float(rmse_h), 'MAE': float(mae_h)}
    }
}

evaluation_file_path = os.path.join(results_dir,
f'final_testing_results_{timestamp}.json')

with open(evaluation_file_path, 'w') as f:
    json.dump(evaluation_results, f, indent=4)
```

Program pada **Tabel 4.60** menyimpan seluruh hasil evaluasi akhir ke dalam file JSON di direktori `EvaluationResults`. Struktur JSON mencakup *timestamp* sesi pengujian, bobot *ensemble* optimal (`best_alpha`, `best_beta`, `best_val_rmse`), serta metrik MSE, RMSE, dan MAE untuk ketiga model. Penyimpanan hasil dalam format JSON memungkinkan hasil evaluasi dipanggil kembali untuk keperluan pelaporan tanpa perlu menjalankan ulang seluruh proses pengujian.